FABRICA DE SOFTWARE & TOPICOS AVANCADOS EM COMPUTACAO
CURSO DE BACHARELADO EM CIENCIA DA COMPUTACAO - TURMA 8MB - CC

RELATORIO TECNICO CUMULATIVO: SPRINTS 01 A 07

SISTEMA QUASE COMPLETO: DASHBOARD, RELATORIOS, FILTROS, BUSCA, EXPORTACAO CSV E AUDITORIA

Consolidado Integral do Projeto SIGAAS: Sistema Integrado de Gestao Academica e Alocacao de Salas com IA

Disciplina:	Fabrica de Software & Topicos Avancados em Computacao
Periodo / Turma:	8o Periodo / Turma 8MB - CC (Semestre 2026/2)
Grupo:	Grupo 31 - SIGAAS
Entrega Academica:	Sprint 07 - Sistema Quase Completo (com Sprints 01 a 06 cumulativas)
Data-Limite:	25 de Outubro de 2026 as 23:59
Repositorio GitHub:	https://github.com/ValdVdC/Fabrica-de-Software-Gerenciador-de-salas

Integrante	Matricula	Papel no Squad	Responsabilidade Tecnica
Ewerton Thyago Tavares da Silva	01573977	Scrum Master (Lider)	Facilitacao agil, governanca do ClickUp e protecao de branches
Wesclei Batista Da Cruz Junior	01606772	Product Owner	Requisitos funcionais, regras de negocio e criterios Gherkin
Osvaldo Vasconcelos de Carvalho	01614171	Dev Backend / Sistemas	FastAPI, servicos de agregacao, CSV streaming e FFI C
Gabriel Porfirio dos Santos	01591399	Dev Frontend	SPA Angular 20, Dashboard SVG, Relatorios e Signals
Flavio Vecch de Brito Farias	01600988	DBA & Documentacao	Modelagem PostgreSQL, migrations Alembic e documentacao

OUTUBRO DE 2026

1. Identificação da Equipe e Distribuição de Papéis

Integrante	Matrícula	Papel no Squad	Responsabilidade Técnica Principal
Ewerton Thyago Tavares da Silva	01573977	Scrum Master (Líder)	Facilitação ágil, gestão de entregas, governança de branches e submissão no Teams
Wesclei Batista Da Cruz Júnior	01606772	Product Owner	Levantamento de requisitos, regras de negócio e validação dos critérios de aceite
Osvaldo Vasconcelos de Carvalho	01614171	Dev Backend / Sistemas	Arquitetura de API, integração FFI C/OpenMP (`ctypes`), endpoints e migrations
Gabriel Porfírio dos Santos	01591399	Dev Frontend	Interface SPA Angular 20, componentes, estados reativos e interceptors
Flávio Vecch de Brito Farias	01600988	DBA & Documentação	Modelagem relacional PostgreSQL, scripts de seed, integridade e documentação técnica

---

PARTE 1 - SPRINT 01: PLANEJAMENTO DO PROJETO (ANTERIOR)

1.1 Escolha do Tema

Desenvolvimento de um sistema multi-campus de gestão acadêmica focado na alocação inteligente de salas e horários, integrando módulos de otimização matemática e inteligência artificial.

1.2 Definição do Problema

A gestão de horários e espaços físicos em instituições de ensino complexas enfrenta desafios que dificultam a operação diária:

• O controle de alocação muitas vezes é ineficiente, direcionando turmas pequenas para salas grandes ou alocando disciplinas práticas em espaços sem os equipamentos adequados.
• A montagem da grade horária é um processo matemático complexo, propício a falhas humanas como o choque de horários entre turmas e professores.
• Existe um desperdício contínuo de espaço físico quando turmas com histórico de alto índice de evasão ou faltas mantêm grandes auditórios ou salas regulares bloqueadas sem real necessidade.
• O remanejamento de salas de última hora gera confusão e não possui um fluxo sistêmico claro de registro e auditoria.
• Isso gera atrasos na liberação de semestres letivos, subutilização de infraestrutura e insatisfação no corpo docente e discente.

1.3 Objetivos do Sistema

• **Objetivo Geral**: Desenvolver uma plataforma multi-tenant que centralize a gestão acadêmica, automatizando a alocação de turmas.
• **Objetivos Específicos**:
• Alocar turmas de forma automática respeitando capacidade, turnos, tipo de sala e necessidade de equipamentos.
• Garantir matematicamente a ausência de conflitos de horários.
• Prever a probabilidade de baixa frequência em aulas específicas utilizando Machine Learning.
• Sugerir o remanejamento proativo de salas baseado na predição da IA, submetendo a ação à aprovação humana.
• Garantir a divisão clara de acessos e permissões por perfil.

1.4 Público-Alvo

A comunidade acadêmica dividida em quatro perfis de acesso: Admin/Coordenação, Secretaria, Professor e Aluno. O uso engloba desde a coordenação (que gerencia e aprova o fluxo pesado) até o aluno final, que consome o resultado do sistema (sua grade de horários).

1.5 Requisitos Funcionais (RF)

Código	Descrição
RF01	Cadastrar, editar e listar campi, salas, equipamentos e cursos.
RF02	Cadastrar disciplinas, criar turmas e registrar a matrícula de alunos.
RF03	Autenticar usuários e redirecionar para interfaces isoladas por perfil de acesso.
RF04	Executar o cálculo de alocação (horário × sala × turma) automaticamente via motor de otimização.
RF05	Registrar a frequência diária dos alunos para alimentar o modelo de IA.
RF06	Gerar sugestões automáticas de remanejamento de sala quando a previsão de faltas for alta.
RF07	Permitir que o perfil Admin/Coordenação aprove ou rejeite as sugestões de remanejamento.
RF08	Consultar a grade de horários formatada por perfil (Minhas aulas como Professor ou Aluno).
RF09	Gerar log e trilha de auditoria para registros de alocação e alterações.

1.6 Requisitos Não Funcionais (RNF)

Código	Descrição
RNF01	Interface web responsiva desenvolvida em Angular.
RNF02	Backend, regras de negócio e orquestração de APIs desenvolvidos em Python (FastAPI*).
RNF03	O motor de alocação deve ser desenvolvido em C, compilado como `.so` e integrado ao Python via `ctypes`.
RNF04	O modelo de Inteligência Artificial preditivo deve utilizar Python com scikit-learn.
RNF05	O banco de dados para persistência relacional e confiável deve ser o PostgreSQL.
RNF06	O controle de versão do código deve ser obrigatoriamente mantido no Git/GitHub.
RNF07	A busca de alocação deve rodar em processamento paralelo para redução de tempo, utilizando OpenMP.

**Nota de Refinamento Técnico*: Conforme previsto nas orientações de arquitetura, o framework backend foi evoluído de Django para FastAPI visando execução assíncrona de alto desempenho, documentação OpenAPI interativa nativa (`/docs`) e comunicação de baixa latência em memória via `ctypes` com o motor em C.

1.7 Casos de Uso Principais

- **UC01**: Realizar Login e rotear por perfil institucional
- **UC02**: Cadastrar Campus, Sala e Curso (Secretaria/Admin)
- **UC03**: Matricular aluno em Turma (Secretaria)
- **UC04**: Executar alocação otimizada de horários (Admin)
- **UC05**: Visualizar grade horária individual (Professor/Aluno)
- **UC06**: Sugerir remanejamento baseado em predição (Sistema/IA)
- **UC07**: Aprovar ou Rejeitar sugestão de remanejamento (Admin/Coordenação)

1.8 Atendimento à Devolutiva da Banca (Sprint 01)

***Parecer da Banca**: "Projeto muito bem estruturado e com diferencial técnico claro. Incluam as datas das sprints e definam métricas para comparar o desempenho sequencial × OpenMP. Coerência: Adequado, proposta muito bem alinhada, com IA, otimização matemática e paralelismo em OpenMP integrados ao problema. Critérios de Entrega: Atendeu, todos os itens foram apresentados, incluindo GitHub; faltam apenas datas no cronograma e métricas para avaliar o ganho do paralelismo."*

1.8.1 Cronograma Oficial com Datas das Sprints

Sprint	Data-Limite	Marco Avaliativo	Escopo Principal no SIGAAS
Sprint 01	Concluída	Planejamento Inicial	Tema, requisitos, formação de equipe e repositório GitHub.
Sprint 02	19/09	Arquitetura e Modelagem	Arquitetura do sistema, Classes, MER, Relacional, Protótipo e Banco criado.
Sprint 03	19/09	Estrutura Inicial Funcional	Conexão PostgreSQL, login com autenticação, perfis RBAC, CRUD e deploy local.
Sprint 04	26/09	Primeiro Módulo Completo	Módulo operacional integrado (Salas, Cursos, Turmas, Matrículas e Grade).
Sprint 05	03/10	Segundo Módulo Funcional	Algoritmo de alocação em C com OpenMP, Speedup e integração `ctypes`.
Sprint 06	17/10	Aprimoramento e IA	Ajustes da Pré-Banca, modelo preditivo scikit-learn e remanejamento.
Sprint 07	24/10	Sistema Quase Completo	Dashboard de indicadores, relatórios, filtros e trilha de auditoria.
Sprint 08	31/10	Sistema Praticamente Concluído	Usabilidade refinada, revisão de regras e permissões multi-campus.
Sprint 09	07/11	Testes Completos	Testes funcionais, validação, cobertura >= 85% e saneamento de bugs.
Sprint 10	14/11	Release Candidate	Sistema estabilizado, OpenAPI/Swagger e README atualizado.
Sprint 11	21/11	Preparação para Entrega	Manual do Usuário, Manual Técnico e code review completo.
Sprint 12	28/11	Versão Final e Vídeos	Code Freeze, produção do vídeo no YouTube (16:9) e Instagram (9:16).
Entrega Final	05/12	Envio Definitivo no Teams	Submissão final sem prorrogação.

1.8.2 Métricas Científicas para Comparação Sequencial × OpenMP

Para atender ao critério de rigor científico de Tópicos Avançados, a avaliação experimental do motor de alocação em C é regida por:

1. Tempo de Execução (T):
• T_s: Tempo sequencial puro com 1 thread.
• $T_p(k)$: Tempo paralelo com $k \in \{1, 2, 4, 8\}$ threads OpenMP.
• Instrumentação via `omp_get_wtime()`, calculando média e desvio padrão em 10 repetições por cenário.

2. Speedup Experimental (S_k):

$$S_k = \frac{T_s}{T_p(k)}$$

3. Eficiência Computacional (E_k):

$$E_k = \frac{S_k}{k} \times 100\%$$

4. Fração Paralelizável Teórica (Lei de Amdahl):

$$S_{\max} = \frac{1}{(1 - f) + \frac{f}{k}}$$

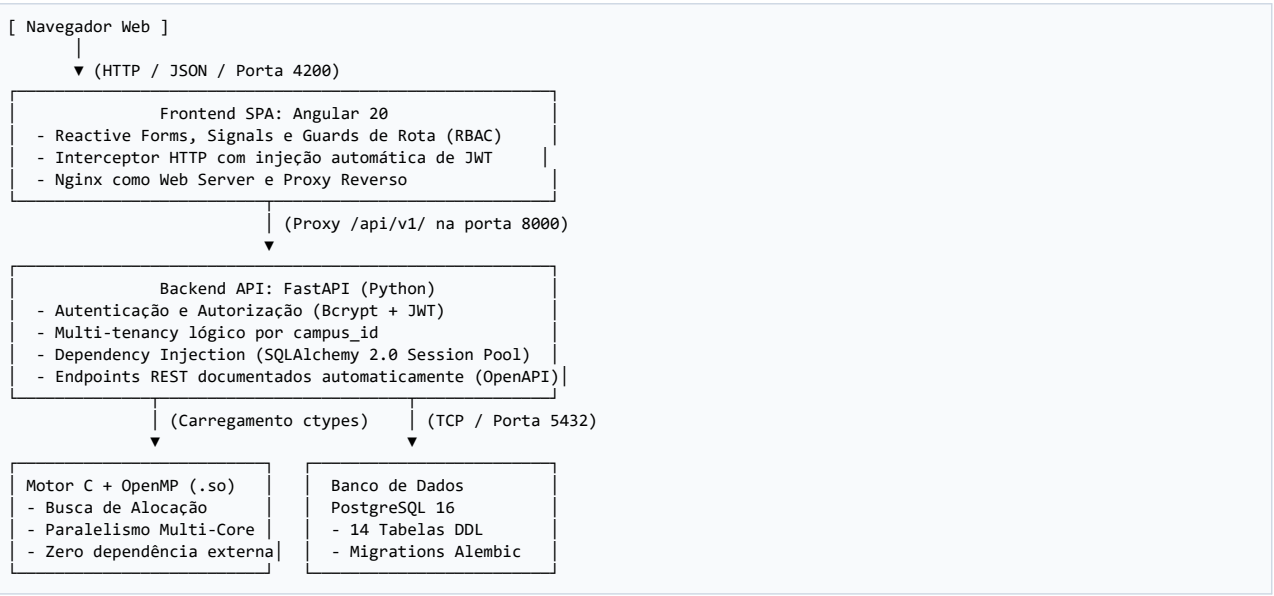
Onde f representa a fração do algoritmo de busca paralelizada entre as threads.

5. Cenários de Carga de Benchmark:
• **Pequeno (Funcional)**: 20 turmas $\times$ 10 salas.
• **Médio (Operacional)**: 100 turmas $\times$ 40 salas com restrições mistas de equipamentos.
• **Stress (Escala Computacional)**: 500 turmas $\times$ 150 salas com restrições densas de turno.

PARTE 2 - SPRINT 02: ARQUITETURA E MODELAGEM DO SISTEMA (ANTERIOR)

2.1 Arquitetura do Sistema

A arquitetura adota separação em camadas desacopladas com isolamento estrito:



2.2 Diagrama de Classes

```
classDiagram
    class Campus {
        +int id
        +string nome
        +string cidade
        +string endereco
        +bool ativo
        +cadastrar()
        +listar_salas()
        +validar_capacidade()
    }

    class Usuario {
        +int id
        +int campus_id
        +string nome
        +string email
        +string senha_hash
        +PerfilUsuario perfil
        +bool ativo
        +autenticar(senha: string) bool
        +gerar_token() string
        +possui_perfil(p: PerfilUsuario) bool
    }

    class Sala {
        +int id
        +int campus_id
        +string bloco
        +string numero
        +TipoSala tipo
        +int capacidade
```

```
        +List~string~ turnos_disponiveis
        +bool ativo
        +verificar_disponibilidade(dia: int, hi: time, hf: time) bool
        +capacidade_suficiente(qtd: int) bool
    }

    class Equipamento {
        +int id
        +string nome
        +string descricao
        +vincular_sala(sala_id: int, qtd: int)
    }

    class SalaEquipamento {
        +int sala_id
        +int equipamento_id
        +int quantidade
        +atualizar_quantidade(qtd: int)
    }

    class Curso {
        +int id
        +int campus_id
        +string nome
        +string codigo
        +adicionar_disciplina(d: Disciplina)
    }

    class Disciplina {
        +int id
        +int curso_id
        +string nome
```

```
        +string codigo
        +int carga_horaria
        +criar_turma(periodo: string) Turma
    }

    class Turma {
        +int id
        +int disciplina_id
        +int professor_id
        +string periodo_letivo
        +int num_matriculados
        +Turno turno_preferido
        +matricular_aluno(u: Usuario) bool
        +calcular_ocupacao() float
    }

    class Matricula {
        +int id
        +int aluno_id
        +int turma_id
        +date data_matricula
        +string status
        +trancar()
        +cancelar()
    }

    class Horario {
        +int id
        +int campus_id
        +int turma_id
        +int sala_id
        +int dia_semana
```

```
        +time hora_inicio
        +time hora_fim
        +detectar_conflito(h: Horario) bool
        +alocar_sala(s: Sala)
    }

    class Frequencia {
        +int id
        +int matricula_id
        +int horario_id
        +date data
        +bool presente
        +marcar_presenca()
        +justificar_falta()
    }

    class PrevisaoFalta {
        +int id
        +int turma_id
        +int horario_id
        +date data_prevista
        +float prob_ausencia
        +string versao_modelo
        +calcular_risco() float
        +exige_remanejamento() bool
    }

    class SugestaoRemanejamento {
        +int id
        +int horario_id
        +int sala_atual_id
        +int sala_sugerida_id
```

```
        +string motivo
        +string justificativa
        +StatusSugestao status
        +int aprovado_por
        +aprovar(coord_id: int)
        +rejeitar(coord_id: int, just: string)
    }

    class LogAlocacao {
        +int id
        +int horario_id
        +TipoEventoLog tipo_evento
        +int usuario_responsavel
        +datetime timestamp
        +string detalhes
        +registrar_evento(tipo: TipoEventoLog, resp_id: int)
    }

    Campus "1" --> "*" Usuario : possui
    Campus "1" --> "*" Sala : possui
    Campus "1" --> "*" Curso : oferta
    Sala "1" --> "*" SalaEquipamento : equipada com
    Equipamento "1" --> "*" SalaEquipamento : alocado em
    Curso "1" --> "*" Disciplina : grade
    Disciplina "1" --> "*" Turma : oferta
    Usuario "1" --> "*" Turma : leciona
    Usuario "1" --> "*" Matricula : cursa
    Turma "1" --> "*" Matricula : contem
    Turma "1" --> "*" Horario : alocada
    Sala "1" --> "*" Horario : sedia
    Horario "1" --> "*" Frequencia : registra
    Horario "1" --> "*" PrevisaoFalta : avaliado por
```

```
    Horario "1" --> "*" SugestaoRemanejamento : remanejado por
    Horario "1" --> "*" LogAlocacao : audita
```

2.3 Modelo Entidade-Relacionamento (MER Conceitual)

```
erDiagram
    CAMPUS {
        int id PK
        string nome
        string cidade
        string endereco
        boolean ativo
    }
    USUARIO {
        int id PK
        int campus_id FK
        string nome
        string email
        string senha_hash
        string perfil
        boolean ativo
    }
    EQUIPAMENTO {
        int id PK
        string nome
        string descricao
    }
    SALA {
        int id PK
        int campus_id FK
        string bloco
        string numero
        string tipo
        int capacidade
        json turnos_disponiveis
        boolean ativo
    }
```

```
    SALA_EQUIPAMENTO {
        int sala_id PK_FK
        int equipamento_id PK_FK
        int quantidade
    }
    CURSO {
        int id PK
        int campus_id FK
        string nome
        string codigo
    }
    DISCIPLINA {
        int id PK
        int curso_id FK
        string nome
        string codigo
        int carga_horaria
    }
    TURMA {
        int id PK
        int disciplina_id FK
        int professor_id FK
        string periodo_letivo
        int num_matriculados
        string turno_preferido
    }
    MATRICULA {
        int id PK
        int aluno_id FK
        int turma_id FK
        date data_matricula
        string status
```

```
    }
    HORARIO {
        int id PK
        int campus_id FK
        int turma_id FK
        int sala_id FK
        int dia_semana
        time hora_inicio
        time hora_fim
    }
    LOG_ALOCACAO {
        int id PK
        int horario_id FK
        json snapshot_evento
        string tipo_evento
        int usuario_responsavel FK
        timestamp timestamp
        text detalhes
    }
    FREQUENCIA {
        int id PK
        int matricula_id FK
        int horario_id FK
        date data
        boolean presente
    }
    PREVISAO_FALTA {
        int id PK
        int turma_id FK
        int horario_id FK
        date data_prevista
        float prob_ausencia
        string versao_modelo
    }
    SUGESTAO_REMANEJAMENTO {
        int id PK
        int horario_id FK
        int sala_atual_id FK
        int sala_sugerida_id FK
        text motivo
        text justificativa
        string status
        int aprovado_por FK
    }

    CAMPUS ||--o{ SALA : "possui"
    CAMPUS ||--o{ USUARIO : "possui"
    CAMPUS ||--o{ CURSO : "oferta"
    SALA ||--o{ SALA_EQUIPAMENTO : "equipada com"
    EQUIPAMENTO ||--o{ SALA_EQUIPAMENTO : "alocado em"
    CURSO ||--o{ DISCIPLINA : "grade curricular"
    DISCIPLINA ||--o{ TURMA : "oferta"
    USUARIO ||--o{ TURMA : "leciona"
    USUARIO ||--o{ MATRICULA : "cursa"
    TURMA ||--o{ MATRICULA : "contem"
    TURMA ||--o{ HORARIO : "alocada em"
    SALA ||--o{ HORARIO : "sedia"
    HORARIO ||--o{ FREQUENCIA : "registra presenca"
    HORARIO ||--o{ PREVISAO_FALTA : "analisada por IA"
    HORARIO ||--o{ SUGESTAO_REMANEJAMENTO : "origina"
    HORARIO ||--o{ LOG_ALOCACAO : "auditada por"
```

2.4 Modelo Relacional

O esquema fisico foi implementado em PostgreSQL com 14 tabelas normalizadas até a Terceira Forma Normal (3FN):

1. `campus` (**id** PK, nome VARCHAR(150), cidade VARCHAR(100), endereco VARCHAR(255), ativo BOOLEAN, created_at TIMESTAMP, updated_at TIMESTAMP)
2. `usuario` (**id** PK, campus_id FK, nome VARCHAR(150), email VARCHAR(255) UNIQUE, senha_hash VARCHAR(255), perfil VARCHAR(30), ativo BOOLEAN, created_at TIMESTAMP, updated_at TIMESTAMP)
3. `equipamento` (**id** PK, nome VARCHAR(100) UNIQUE, descricao VARCHAR(255), created_at TIMESTAMP)
4. `sala` (**id** PK, campus_id FK, bloco VARCHAR(50), numero VARCHAR(50), tipo VARCHAR(30), capacidade INT, turnos_disponiveis JSON, ativo BOOLEAN, created_at TIMESTAMP, updated_at TIMESTAMP, UNIQUE(campus_id, bloco, numero), CHECK(capacidade > 0))
5. `sala_equipamento` (**sala_id** PK/FK, **equipamento_id** PK/FK, quantidade INT, CHECK(quantidade > 0))
6. `curso` (**id** PK, campus_id FK, nome VARCHAR(150), codigo VARCHAR(30), created_at TIMESTAMP, updated_at TIMESTAMP, UNIQUE(campus_id, codigo))

7. `disciplina` (**id** PK, curso_id FK, nome VARCHAR(150), codigo VARCHAR(30), carga_horaria INT, created_at TIMESTAMP, updated_at TIMESTAMP, UNIQUE(curso_id, codigo), CHECK(carga_horaria > 0))
8. `turma` (**id** PK, disciplina_id FK, professor_id FK, periodo_letivo VARCHAR(20), num_matriculados INT, turno_preferido VARCHAR(20), created_at TIMESTAMP, updated_at TIMESTAMP, CHECK(num_matriculados >= 0))
9. `matricula` (**id** PK, aluno_id FK, turma_id FK, data_matricula DATE, status VARCHAR(20), created_at TIMESTAMP, UNIQUE(aluno_id, turma_id))
10. `horario` (**id** PK, campus_id FK, turma_id FK, sala_id FK, dia_semana SMALLINT, hora_inicio TIME, hora_fim TIME, created_at TIMESTAMP, updated_at TIMESTAMP, CHECK(hora_inicio < hora_fim), CHECK(dia_semana >= 0 AND dia_semana <= 6))
11. `log_alocacao` (**id** PK, horario_id FK, snapshot_evento JSON, tipo_evento VARCHAR(50), usuario_responsavel FK, timestamp TIMESTAMP, detalhes TEXT)
12. `frequencia` (**id** PK, matricula_id FK, horario_id FK, data DATE, presente BOOLEAN, created_at TIMESTAMP)
13. `previsao_falta` (**id** PK, turma_id FK, horario_id FK, data_prevista DATE, prob_ausencia FLOAT, versao_modelo VARCHAR(50), criado_em TIMESTAMP)
14. `sugestao_remanejamento` (**id** PK, horario_id FK, sala_atual_id FK, sala_sugerida_id FK, motivo TEXT, justificativa TEXT, status VARCHAR(20), aprovado_por FK, criado_em TIMESTAMP, atualizado_em TIMESTAMP)

2.5 Protótipos das Telas Principais

A interface web foi concebida por perfil de acesso no padrão design system institucional:

1. Tela de Autenticação (`/login`): Formulário centralizado com validação em tempo real e botões de atalho de demonstração para os perfis.
2. Painel do Administrador (`/admin`): Sistema em abas contextuais ("Salas & Espaços", "Equipamentos", "Resumo & Métricas"), com operações completas de criação, edição e exclusão.
3. Painel da Secretaria (`/secretaria`): Gestão de ofertas com 5 abas integradas ("Cursos", "Disciplinas", "Turmas", "Matrículas", "Métricas").
4. Painel do Docente (`/professor`): Grade Horária Semanal formatada em matriz visual de segunda a sábado com destaque para código da disciplina, bloco e sala alocada.
5. Painel do Discente (`/aluno`): Grade Horária personalizada com localização de salas e turnos.

2.6 Banco de Dados Criado

O banco de dados encontra-se plenamente criado através do script de migração versionada do Alembic (`backend/alembic/versions/001_initial_schema.py`), executado no PostgreSQL 16 com integridade referencial rigorosa.

2.7 Projeto Estruturado no GitHub

O repositório oficial adota **Trunk-Based Development**, governança de commits convencionais e CI em 3 estágios:

• **URL**: https://github.com/ValdVdC/Fabrica-de-Software-Gerenciador-de-salas
• **Política de Branches**: Nenhuma alteração é enviada diretamente à `main`. Todo trabalho transita por branches semânticas (`feat/*`, `docs/*`, `chore/*`) e Pull Requests avaliados pela esteira automatizada de CI.
• **Squash and Merge Exclusivo**: Repositório travado com `allow_merge_commit=false` e `allow_rebase_merge=false`, garantindo histórico linear e atômico.

PARTE 3 - SPRINT 03: ESTRUTURA INICIAL FUNCIONAL (ANTERIOR)

3.1 Banco de Dados Conectado

A integração entre o FastAPI e o PostgreSQL é realizada via SQLAlchemy 2.0 com engine síncrona robusta (`create_engine`) e pool de conexões resilientes com validação ativa (`pool_pre_ping=True`), configurado em `backend/app/db/session.py`.

• **Evidência de Infraestrutura**: Na inicialização dos containers, o serviço `sigaas_postgres` valida a prontidão através do healthcheck `pg_isready -U gestao_salas -d gestao_salas`.
• **Verificação Ativa via Endpoint HTTP**: O endpoint `GET /health` executa ativamente uma query (`SELECT 1`) na sessão do banco via injeção de dependência `get_db`, retornando o payload JSON:

```
{
  "status": "ok",
  "service": "SIGAAS API",
  "version": "0.1.0",
  "database": "connected"
}
```

• **Comprovação Fim a Fim**: A conectividade funcional com o banco é comprovada de ponta a ponta pelas operações reais de consulta e escrita executadas no login (`/auth/login`), nos endpoints de CRUD (`/api/v1/salas`, `/api/v1/usuarios`), nas migrações do Alembic e no script de seed.

3.2 Login Funcional

A autenticação foi implementada em conformidade com o padrão OAuth2 e recomendações OWASP para APIs REST:

• **Endpoint**: `POST /api/v1/auth/login`
• **Payload**: JSON contendo `{"email": "...", "senha": "..."}`
• **Segurança**: Criptografia de senhas via hash `Bcrypt` com salt aleatório (nunca gravando texto plano).
• **Resposta**: Retorno do token `access_token` assinado digitalmente via JWT (`HS256`, validade de 60 minutos) acompanhado dos dados do usuário (`id`, `nome`, `email`, `perfil`, `campus_id`).
• **Endpoint de Sessão e Esquema de Segurança**: `GET /api/v1/auth/me` protegido pelo esquema de autenticação `OAuth2PasswordBearer(tokenUrl="/api/v1/auth/login")` com validação de cabeçalho `Authorization: Bearer <token>`, permitindo ao cliente Angular revalidar a sessão ativa.

3.3 Cadastro de Usuários e Persistência

A gestão de usuários e criação de contas institucionais operam com persistência no PostgreSQL e salvaguardas rigorosas de autenticação:

1. Endpoint Operacional de Cadastro (`POST /api/v1/usuarios`):
• **Autorização RBAC**: Restrito aos perfis `admin`, `coordenador` e `secretaria` via injeção `require_role`.
• **Criptografia Segura**: Criptografia imediata da senha via algoritmo `Bcrypt` com geração de salt aleatório.
• **Validações de Domínio e Formato**: Verificação estrita de formato de e-mail institucional via schema Pydantic v2 `EmailStr` (rejeitando strings inválidas com código `HTTP 422 Unprocessable Entity`), validação de integridade referencial do `campus_id` associado e bloqueio de unicidade de e-mail duplicado com retorno `HTTP 409 Conflict`.
• **Proteção de Dados**: Retorno via schema Pydantic `UsuarioRead`, suprimindo o campo de senha da resposta.
2. Carga Inicial para Homologação e Testes (Seed Idempotente):
• Script automatizado `scripts/seed_db.py` provisiona 9 usuários de demonstração no banco de dados cobrindo todos os perfis institucionais:
• Administrador: `admin@sigaas.edu` (senha: `sigaas123`)
• Coordenador: `coord.cc@sigaas.edu` (senha: `sigaas123`)
• Secretaria: `secretaria@sigaas.edu` (senha: `sigaas123`)
• Professores: `alan.turing@sigaas.edu`, `ada.lovelace@sigaas.edu`, `professor@sigaas.edu` (senha: `sigaas123`)
• Alunos: `joao.silva@sigaas.edu`, `maria.souza@sigaas.edu`, `aluno@sigaas.edu` (senha: `sigaas123`)

3.4 Controle de Perfis de Acesso (RBAC)

O controle de privilégios opera em dois níveis sincronizados:

1. Backend (FastAPI): As rotas operacionais utilizam a fábrica de dependência `require_role(PerfilUsuario.ADMIN, PerfilUsuario.COORDENADOR)` em `backend/app/api/deps.py`, que recebe os perfis autorizados como argumentos posicionais e valida o perfil do usuário extraído do token JWT autenticado. Tentativas por perfil não autorizado resultam em código `HTTP 403 Forbidden`.
2. Frontend (Angular): O guarda de rotas `RoleGuard` (`frontend/src/app/guards/role.guard.ts`) utiliza o serviço `AuthService.hasRole()` baseado nos dados do perfil retornados pelo endpoint de login e mantidos na sessão do cliente, redirecionando o usuário para a interface correspondente ao seu perfil.

3.5 CRUD Principal Funcionando com Dados Reais

O SIGAAS implementa o ciclo das quatro operações fundamentais de persistência (**Cadastrar, Consultar, Atualizar e Excluir**) na entidade principal de gestão física (`Sala`), conectado ao PostgreSQL sob isolamento multi-tenant:

1. Cadastrar (Create): `POST /api/v1/salas` com validação de unicidade de bloco e número por campus (`HTTP 409 Conflict`), capacidade entre 1 e 5000 alunos e turnos disponíveis.
2. Consultar (Read): `GET /api/v1/salas` com paginação e filtro automático pelo campus do usuário.
3. Atualizar (Update): `PUT /api/v1/salas/{sala_id}` para Administrador e Coordenador, com atualização atômica de bloco, número, tipo, capacidade e status ativo.
4. Excluir (Delete): `DELETE /api/v1/salas/{sala_id}` restrito exclusivamente ao Administrador, bloqueando exclusão com erro `HTTP 409 Conflict` se houver horários de aula alocados para a sala.

3.6 Procedimento de Execução Local (Deploy em 1 Comando)

O ecossistema completo roda em containers Docker interligados em uma rede bridge privada (`sigaas_network`):

```
# 1. Clonar o repositório
git clone https://github.com/ValdVdC/Fabrica-de-Software-Gerenciador-de-salas.git
cd Fabrica-de-Software-Gerenciador-de-salas

# 2. Inicializar os containers (PostgreSQL, Backend FastAPI, Frontend Nginx e Cron)
docker compose up -d --build

# 3. Executar as migrações do banco de dados (Alembic)
docker compose exec backend alembic upgrade head

# 4. Popular os dados de teste (Seed)
# Opção A — Execução dentro do container (com volume /scripts montado e PYTHONPATH=/app):
docker compose exec -e PYTHONPATH=/app backend python /scripts/seed_db.py

# Opção B — Execução a partir do host (com banco do Docker ativo):
python scripts/seed_db.py
```

URLs de Acesso no Navegador:

- **Frontend SPA (Angular)**: `http://localhost:4200`
- **Documentação Interativa da API (Swagger UI)**: `http://localhost:4200/docs` ou `http://localhost:8000/docs`
- **Status do Motor de Alocação C**: `http://localhost:8000/api/v1/motor/status`

3.7 Dificuldades Encontradas e Soluções Aplicadas na Sprint 03

1. Compilação C/OpenMP no Container Slim:
• *Desafio*: O container Python 3.12 Slim não incluía os cabeçalhos padrão da biblioteca C (`stdio.h`, `libc6-dev`) para compilar `motor_alocacao.so`.
• *Solução*: Substituição de pacotes avulsos pelo metapacote `build-essential` no `Dockerfile` do backend, garantindo compilação nativa com OpenMP.
2. Normalização de Final de Linha (CRLF vs LF) e Cache Docker:
• *Desafio*: Ambientes Windows geravam quebras de linha CRLF que causavam falhas em scripts shell montados em containers Linux.
• *Solução*: Configuração de arquivo `.gitattributes` forçando `eol=lf` em scripts e adição de diretivas no Dockerfile.
3. Limite de Diff por PR:
• *Desafio*: Manter cada entrega dentro do limite normativo de no máximo 400 linhas de código modificado por PR.
• *Solução*: Fatiamento vertical estrito com criação de serviços reativos no frontend em PRs complementares aos endpoints de backend, acompanhado de testes de contrato automatizados.

3.8 Próximos Passos (Transição para Sprint 04 e 05)

1. Primeiro Módulo Completo da Sprint 04: Integração completa dos módulos de cadastro acadêmico e visualização de grade horária (Salas, Cursos, Turmas, Matrículas e Horários).
2. Desenvolvimento do Algoritmo de Alocação em C (Sprint 05): Implementação do algoritmo exato/heurístico de busca de salas sem conflitos em `backend/motor_alocacao/motor.c`.
3. Paralelização OpenMP e Coleta Científica de Speedup: Medição dos tempos de execução sequencial vs. paralelo com $k \in \{1, 2, 4, 8\}$ threads sob os cenários de carga de 20, 100 e 500 turmas.

3.9 Registro e Justificativa de Ajustes de Modelagem (Sprint 02 → Sprint 03)

Em atendimento às boas práticas de engenharia de software e aos critérios avaliativos da disciplina, registra-se a evolução técnica entre a modelagem conceitual inicial (Sprint 02) e o modelo físico relacional implementado e versionado via Alembic (Sprint 03):

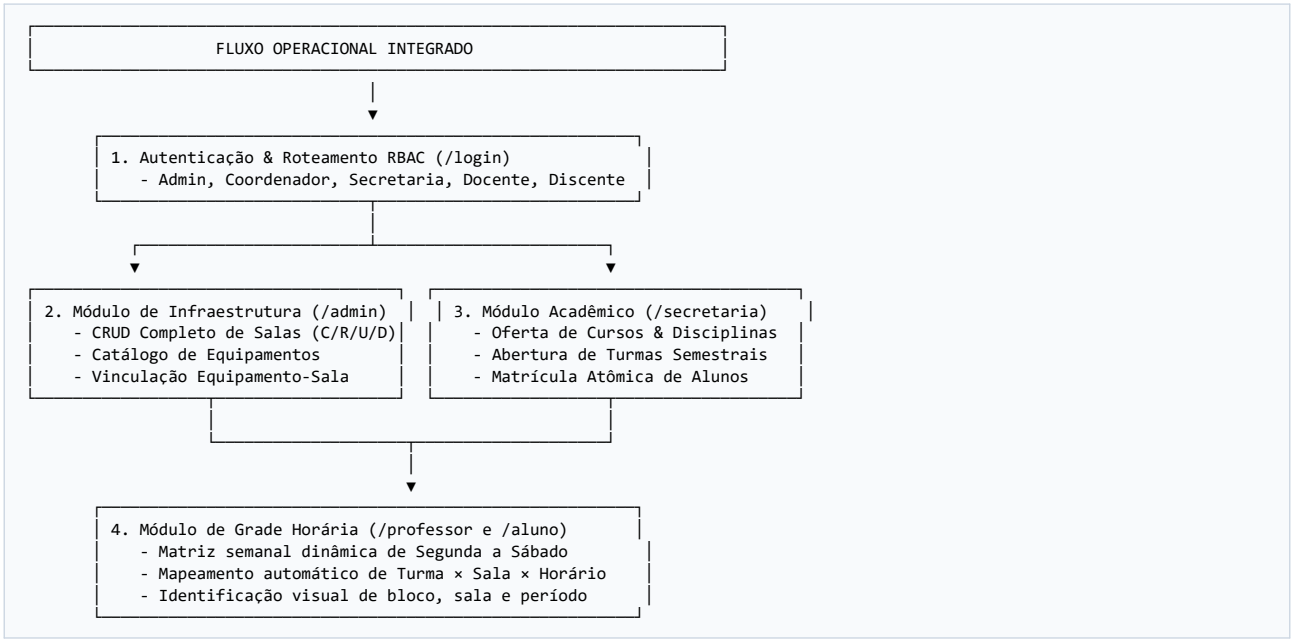
Item de Modelagem	Estado Conceitual (Sprint 02)	Estado Físico Refinado (Sprint 03)	Justificativa Técnica da Alteração
Nomenclatura de Tabelas	Plural (`campi`, `usuarios`, `salas`)	Singular (`campus`, `usuario`, `sala`)	Padronização idiomática em PostgreSQL e conformidade direta com os nomes de classes do ORM SQLAlchemy 2.0.
Auditoria Temporal	Campo único `criado_em`	Par atômico `created_at` e `updated_at`	Rastreabilidade granular do ciclo de vida das entidades, permitindo identificar quando um registro foi modificado após sua criação.
Tabela Associativa	`sala_equipamento` com ID substituto (`id` PK)	Chave primária composta (`sala_id`, `equipamento_id`)	Eliminação de surrogate key desnecessária; garantia estrutural e física no SGBD contra duplicidade de associação entre sala e equipamento.
Ciclo de Vida / Soft-Delete	Não contemplado em `campus` e `sala`	Coluna booleana `ativo` com valor padrão `true`	Preservação de integridade referencial com horários históricos sem exclusão física de registros essenciais.

Item de Modelagem	Estado Conceitual (Sprint 02)	Estado Físico Refinado (Sprint 03)	Justificativa Técnica da Alteração
Tipagem de Enumerações	Tipos nativos `CREATE TYPE ... AS ENUM`	Colunas `VARCHAR` com restrições `CHECK` e Pydantic	Portabilidade em migrações, redução de overhead de locking em DDLs no PostgreSQL e validação semântica em nível de aplicação.
Rastreamento de Matrícula	Apenas timestamp de matrícula	Adição de coluna `status: VARCHAR(20)` (`ativa`, `trancada`, `cancelada`)	Viabilização do controle acadêmico de evasão discente que alimenta o modelo preditivo de faltas.
Integridade de Alocação	Restrições básicas	`CHECK(hora_inicio < hora_fim)` e `CHECK(dia_semana BETWEEN 0 AND 6)`	Salvaguarda física contra corrupção de horários no banco antes mesmo do motor em C processar a grade.

PARTE 4 - SPRINT 04: PRIMEIRO MÓDULO COMPLETO (ANTERIOR)

4.1 Descrição do Primeiro Módulo Totalmente Funcional

A Sprint 04 consolida o **Módulo Operacional de Gestão de Infraestrutura e Oferta Acadêmica**, integrando ponta a ponta as interfaces gráficas da SPA Angular, a camada de serviços RESTful do FastAPI e o armazenamento relacional no PostgreSQL 16. O sistema opera com fluxos reais de ponta a ponta, sem dados estáticos ou telas fictícias (*mockadas*).



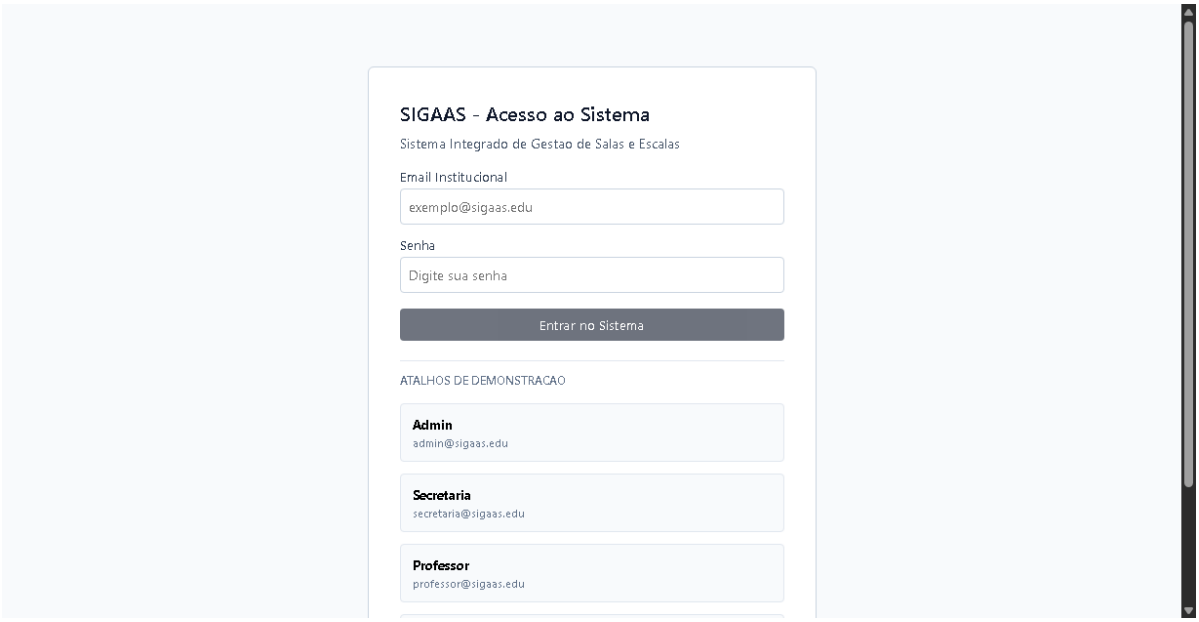


Figura 1 - Tela de login com contas demonstrativas para alternância rápida entre perfis RBAC.

4.1.1 Gestão de Salas e Espaços Físicos (Painel do Administrador)

A interface de infraestrutura (`/admin`) permite ao Administrador e Coordenador executar o ciclo de vida completo dos espaços físicos:

1. Cadastro (Create): Formulário interativo para definição de Bloco, Número, Tipo (Regular, Laboratório, Auditório, Reunião) e Capacidade física (1 a 5000 postos).
2. Consulta (Read): Tabela reativa com paginação, badges semânticos de tipo de espaço, capacidade instalada e turnos disponíveis.
3. Atualização (Update): Modo de edição com formulário contextual para ajuste de capacidade, tipo, bloco ou número, chamando `PUT /api/v1/salas/{id}` com recálculo de integridade e resposta imediata na tabela.
4. Exclusão (Delete): Remoção direta via `DELETE /api/v1/salas/{id}` com salvaguarda física (bloqueio automático caso existam horários de aulas alocados para o espaço) e feedback visual claro.

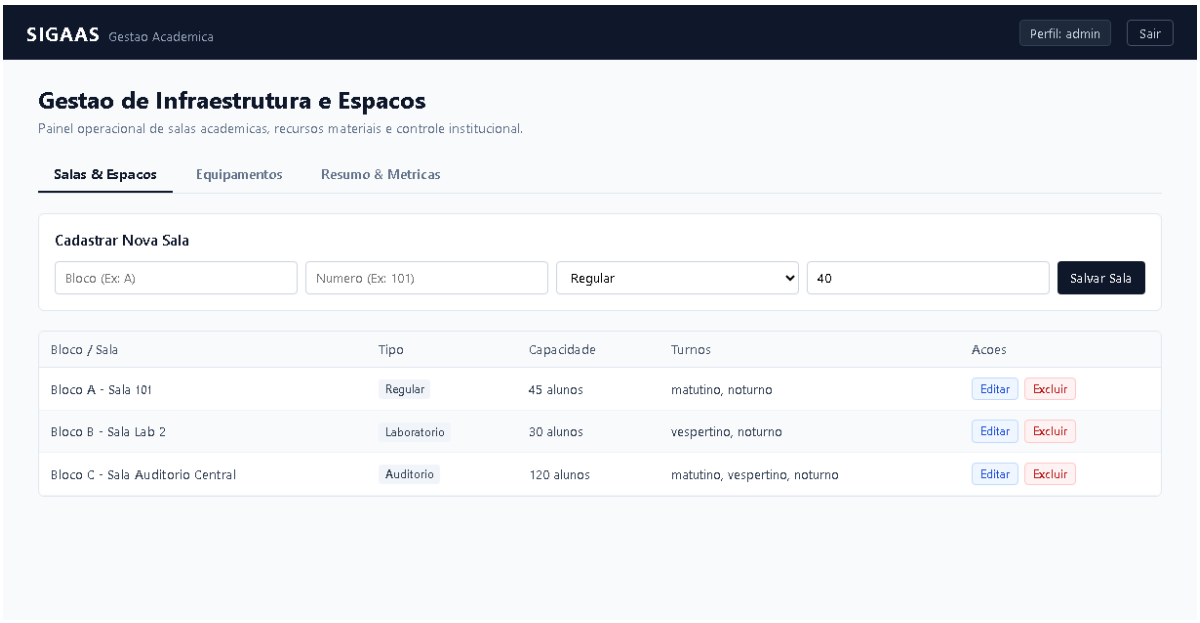


Figura 2 - Consulta reativa de salas físicas com paginação, badges semânticos de tipo e ações operacionais de edição e exclusão.

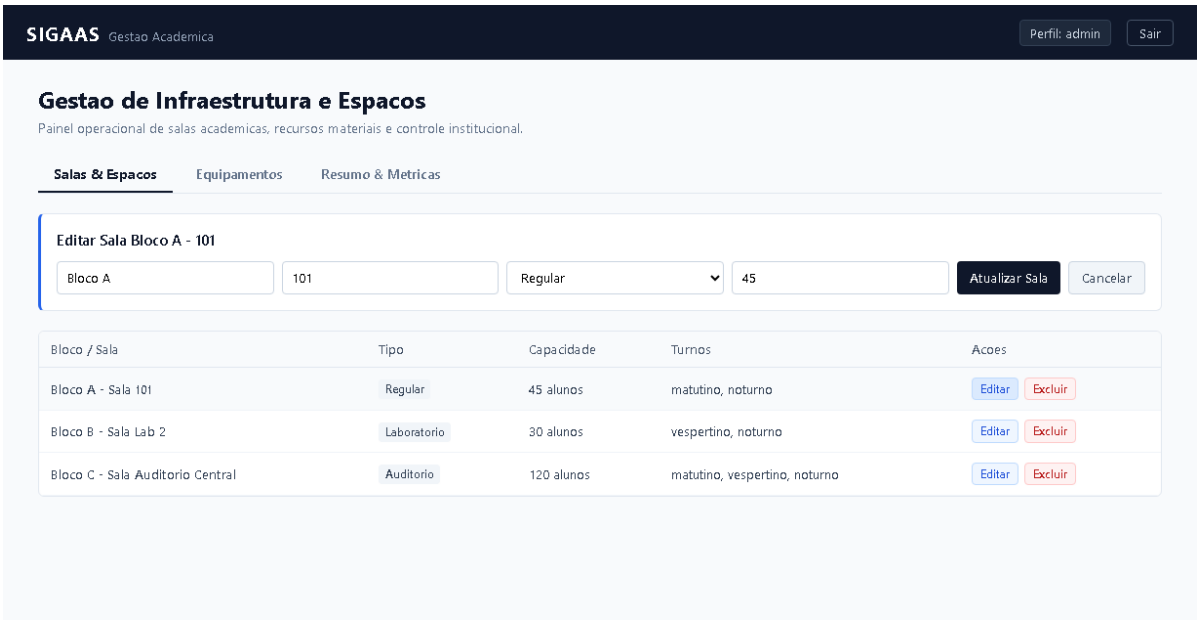


Figura 3 - Modo de edição de sala física com autopreenchimento e validação de capacidade.

4.1.2 Catálogo e Associação de Recursos Materiais

A aba de Equipamentos permite o cadastro padronizado de recursos (ex.: Projetores, Kits de Robótica, Terminais de Laboratório) e o formulário visual de **Associação Equipamento-Sala**, conectando o recurso material à sala com especificação de quantidade

física disponível.

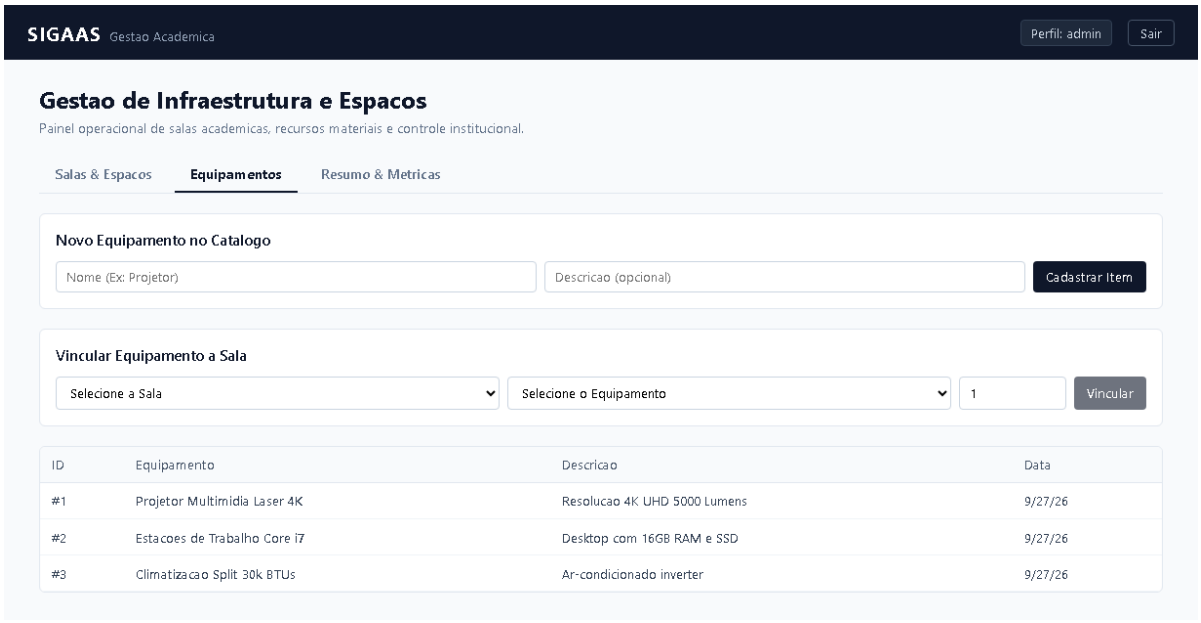


Figura 4 - Catálogo de equipamentos e formulário de vinculação de recursos materiais às salas com especificação de quantidade instalada.

4.1.3 Gestão Acadêmica de Cursos, Disciplinas, Turmas e Matrículas (Secretaria)

No painel da Secretaria (`/secretaria`), o fluxo acadêmico é composto de forma hierárquica e encadeada:

1. Cursos: Cadastro institucional com código único por campus (ex.: `ENG` para Engenharia).
2. Disciplinas: Vinculadas ao curso de origem com carga horária em horas.
3. Turmas: Abertura de turmas para o período letivo (ex.: `2026.1`), com definição de turno preferido (`matutino`, `vespertino`, `noturno`, `integral`) e atribuição de docente responsável.
4. Matrículas: Inscrição de discentes na turma correspondente, com validação de duplicidade e incremento atômico do contador `num_matriculados`.

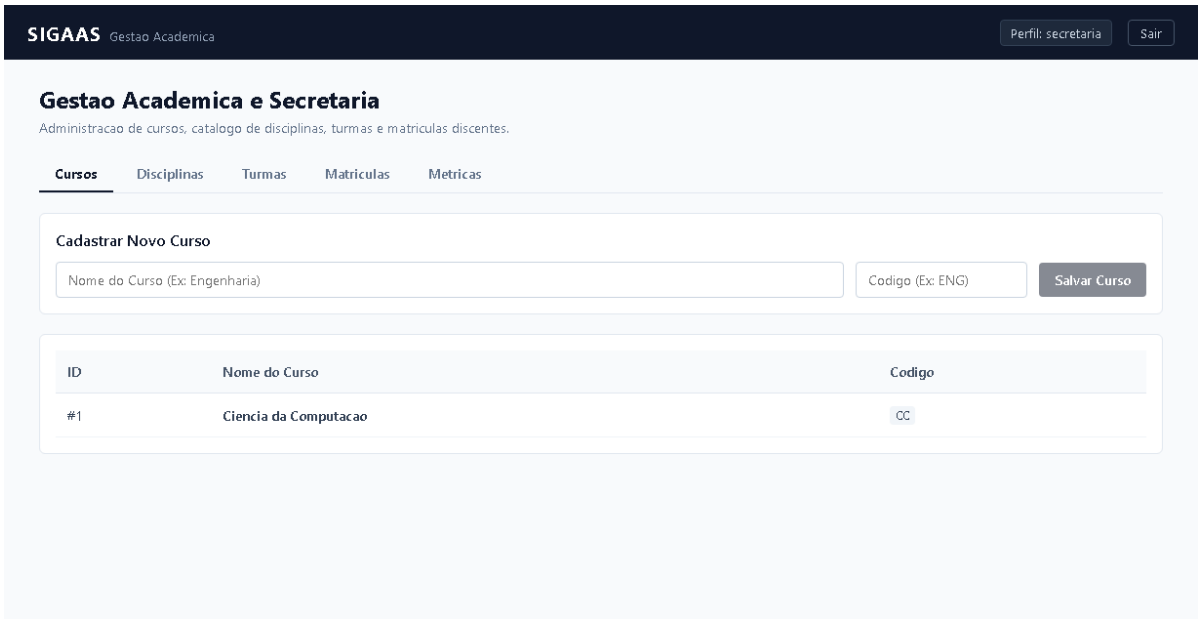


Figura 5 - Gestão integrada de cursos, disciplinas curriculares, turmas semestrais e matrículas discentes.

4.1.4 Grade Horária Semanal Integrada (Docentes e Discentes)

O componente `GradeHorariaComponent` consome o endpoint `GET /api/v1/horarios/meus`:

• **Docentes (`/professor`)**: Visualizam a grade semanal das turmas sob sua regência, identificando a sala e bloco onde ministrarão cada aula.
• **Discentes (`/aluno`)**: Visualizam a grade personalizada correspondente às suas matrículas ativas, com horários de início e término e localização exata no campus.

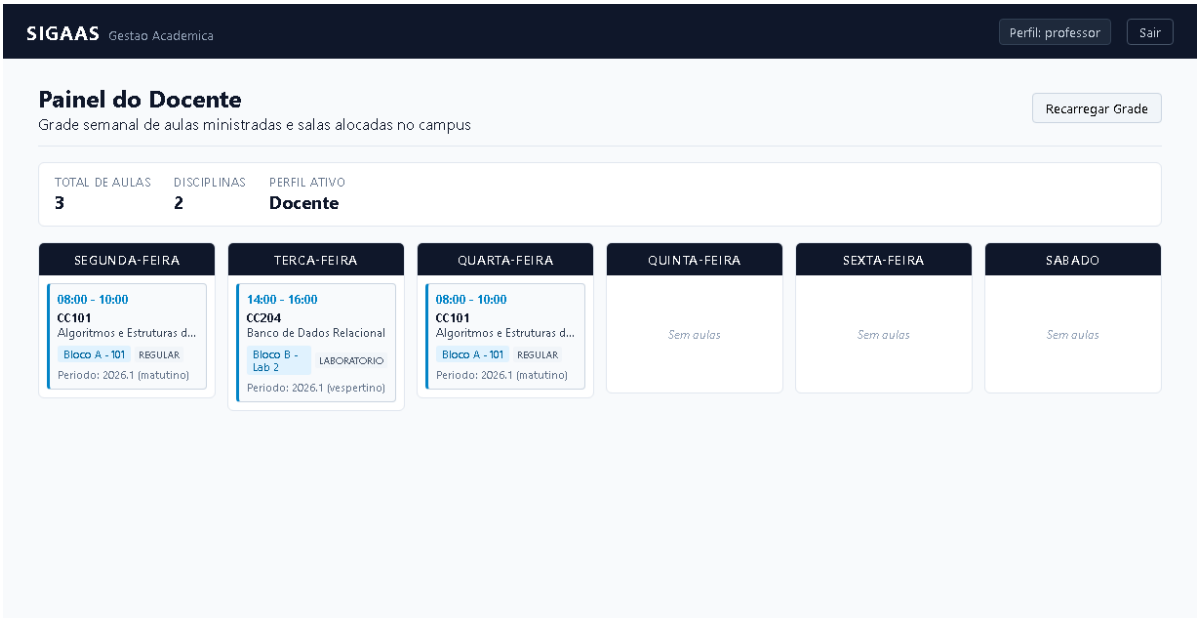


Figura 6 - Matriz visual de horários semanais do professor com identificação de turma, disciplina e sala alocada.

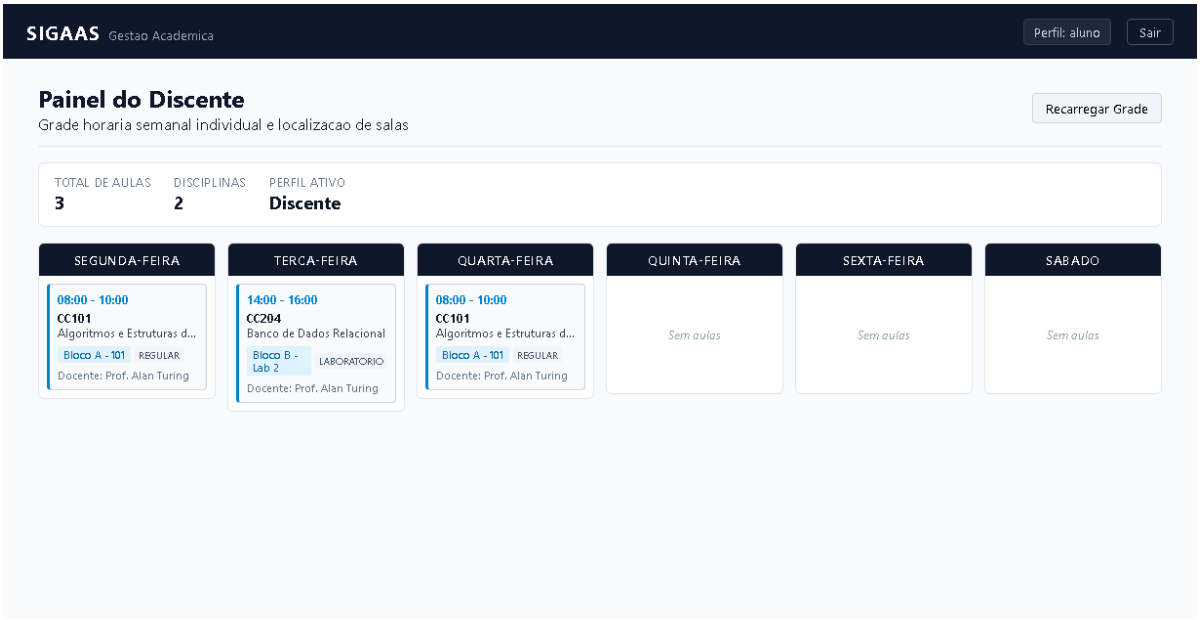


Figura 7 - Visão semanal personalizada do discente exibindo turmas matriculadas e salas alocadas.

4.2 Persistência de Dados e Evidências no PostgreSQL 16

O módulo opera conectado ao banco relacional PostgreSQL 16 Alpine, com persistência assegurada através de volume gerenciado Docker (`postgres_data`). Todas as operações realizadas na interface web geram transações SQL imediatas com isolamento ACID.

4.2.1 Sobrevivência a Reinicializações

A durabilidade dos dados foi validada através do ciclo de derrubada e reinicialização completa dos contêineres:

```
docker compose down
docker compose up -d
```

Após o reinício dos serviços, todas as salas cadastradas, turmas abertas e matrículas discentes permanecem intactas no banco, confirmando a correta montagem do volume persistente.

4.2.2 Evidências de Consultas SQL Reais no PostgreSQL

Abaixo constam os extratos diretos extraídos da base de dados operacional:

1. Consulta de Salas e Capacidades (`SELECT * FROM sala`):

```
SELECT id, campus_id, bloco, numero, tipo, capacidade, turnos_disponiveis, ativo
FROM sala
WHERE campus_id = 1
ORDER BY bloco, numero;
```

```
 id | campus_id | bloco | numero |    tipo     | capacidade | turnos_disponiveis        | ativo
----+-----------+-------+--------+-------------+------------+---------------------------+-------
  1 |         1 | A     | 101    | regular     |         45 | ["matutino", "noturno"]   | t
  2 |         1 | A     | 102    | regular     |         40 | ["matutino", "vespertino"]| t
  3 |         1 | B     | Lab1   | laboratorio |         30 | ["matutino", "vespertino"]| t
  4 |         1 | C     | Aud1   | auditorio   |        120 | ["integral"]              | t
(4 rows)
```

2. Consulta de Turmas e Ocupação (`SELECT * FROM turma`):

```
SELECT t.id, d.codigo AS disciplina, t.periodo_letivo, t.turno_preferido, t.num_matriculados, u.nome AS professor
FROM turma t
JOIN disciplina d ON d.id = t.disciplina_id
LEFT JOIN usuario u ON u.id = t.professor_id
WHERE d.curso_id IN (SELECT id FROM curso WHERE campus_id = 1);
```

```
 id | disciplina | periodo_letivo | turno_preferido | num_matriculados |    professor
----+------------+----------------+-----------------+------------------+-----------------
  1 | CC101      | 2026.1         | matutino        |                2 | Alan Turing
  2 | CC204      | 2026.1         | matutino        |                1 | Ada Lovelace
(2 rows)
```

3. Consulta de Matrículas Discentes (`SELECT * FROM matricula`):

```
SELECT m.id, u.nome AS aluno, d.nome AS disciplina, m.data_matricula, m.status
FROM matricula m
JOIN usuario u ON u.id = m.aluno_id
JOIN turma t ON t.id = m.turma_id
JOIN disciplina d ON d.id = t.disciplina_id
ORDER BY m.id;
```

```
 id |    aluno    |    disciplina     | data_matricula | status
----+-------------+-------------------+----------------+--------
  1 | Joao Silva  | Algoritmos I      | 2026-03-01     | ativa
  2 | Maria Souza | Algoritmos I      | 2026-03-02     | ativa
  3 | Joao Silva  | Bancos de Dados I | 2026-03-01     | ativa
(3 rows)
```

4. Consulta de Alocações de Grade Horária (`SELECT * FROM horario`):

```
SELECT h.id, d.codigo AS disciplina, s.bloco || '-' || s.numero AS sala,
       h.dia_semana, h.hora_inicio, h.hora_fim
FROM horario h
JOIN turma t ON t.id = h.turma_id
JOIN disciplina d ON d.id = t.disciplina_id
JOIN sala s ON s.id = h.sala_id
ORDER BY h.dia_semana, h.hora_inicio;
```

```
 id | disciplina |  sala   | dia_semana | hora_inicio | hora_fim
----+------------+---------+------------+-------------+----------
  1 | CC101      | A-101   |          0 | 08:00:00    | 10:00:00
  2 | CC101      | A-101   |          2 | 08:00:00    | 10:00:00
  3 | CC204      | B-Lab1  |          1 | 10:00:00    | 12:00:00
(3 rows)
```

4.3 Catálogo de Validações e Regras de Negócio Implementadas

O sistema aplica arquitetura de validação defensiva em camadas duplas: **validação antecipada no Frontend** (agilidade e feedback ao usuário) e **validação inviolável no Backend / Banco de Dados** (garantia de integridade e segurança):

Regra de Negócio	Camada de Frontend	Camada de Backend (FastAPI / SQLAlchemy)	Código HTTP / Comportamento
Unicidade de Sala por Campus	Bloqueio de envio com campos vazios	`SELECT Sala WHERE campus_id AND bloco AND numero` + `UNIQUE(campus_id, bloco, numero)`	`409 Conflict`
Limites de Capacidade de Sala	Campo numérico `min="1" max="5000"`	Pydantic `Field(ge=1, le=5000)` + `CHECK (capacidade > 0)`	`422 Unprocessable`
Formato Estrito de E-mail	Input `type="email"` com regex RFC 5322	Pydantic v2 `EmailStr` com normalização de domínio	`422 Unprocessable`
Unicidade de Matrícula Discente	Desabilitação de botão após submissão	`UNIQUE(aluno_id, turma_id)` no banco + verificação de vínculo	`409 Conflict`
Incremento Atômico de Matrículas	Atualização reativa de signals	`turma.num_matriculados += 1` em transação atômica ACID	`201 Created`
Proteção de Exclusão de Sala	Diálogo de ação com botão explícito	Verificação de chave estrangeira em `horarios.sala_id`	`409 Conflict`
Isolamento Multi-Campus (Tenant)	Campus extraído da sessão JWT ativa	`WHERE campus_id == current_user.campus_id` nas queries	`403 Forbidden`
Carga Horária Positiva	Input numérico `min="1"`	Pydantic `Field(gt=0)` + `CHECK (carga_horaria > 0)`	`422 Unprocessable`
Consistência de Intervalo de Aula	Formatação com 2 dígitos (`08:00`)	`CHECK (hora_inicio < hora_fim)` e `CHECK (dia_semana BETWEEN 0 AND 6)`	`422 Unprocessable`

4.4 Tratamento de Erros e Feedback ao Usuário

O sistema implementa uma camada padronizada de captura de exceções para garantir que o usuário nunca seja exposto a telas em branco, mensagens técnicas ininteligíveis ou falhas silenciosas:

1. Camada de Interceptors e Serviços Reativos (`AdminService`, `SecretariaService`, `HorarioService`):
• Todas as requisições HTTP são monitoradas pelo signal `isLoading`, que desabilita botões de submissão e exibe indicadores visuais para impedir cliques duplos e condições de corrida (*race conditions*).
2. Tratamento Semântico de Erros HTTP:
• **401 Unauthorized**: Limpeza da sessão do usuário e redirecionamento suave para `/login` com banner explicativo.
• **403 Forbidden**: Banner visual alertando ausência de privilégios para o recurso requisitado.
• **409 Conflict**: Mensagem contextual clara indicando duplicidade (ex.: *"Ja existe sala no bloco 'A' e numero '101' para este campus"* ou *"Nao e possivel excluir sala com horarios alocados"*).
• **422 Unprocessable Entity**: Extração das mensagens de validação do Pydantic, convertendo arrays de erro em avisos legíveis.
• **500 Internal Server Error**: Sanitização estrita. O backend jamais vaza *stacktraces* ou strings de banco; a UI exibe *"Erro interno no servidor. Tente novamente mais tarde."*
3. Padrão Visual de Alertas:
• **Sucesso**: Caixa com borda verde suave (`#bbf7d0`), fundo verde claro (`#f0fdf4`) e texto verde escuro (`#15803d`).
• **Erro**: Caixa com borda vermelha (`#fecaca`), fundo vermelho claro (`#fef2f2`) e texto vermelho escuro (`#b91c1c`).

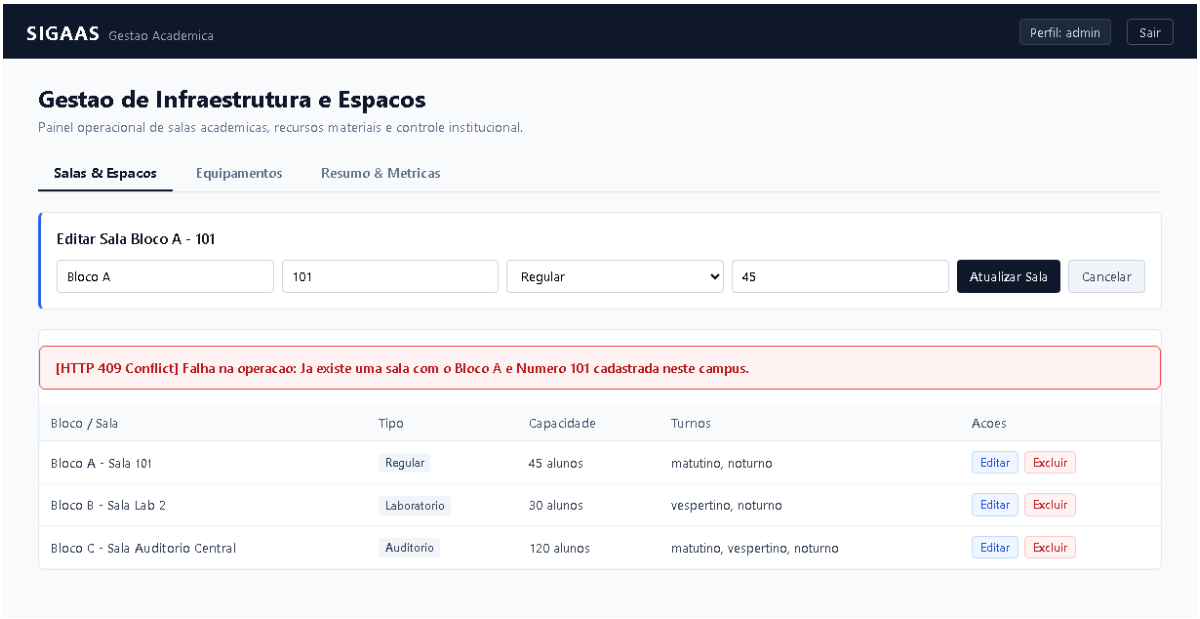


Figura 8 - Notificação semântica de conflito na interface ao tentar cadastrar sala duplicada no mesmo campus.

4.5 Navegação, Usabilidade e Acessibilidade

4.5.1 Diagrama de Navegação entre Telas

O fluxo de telas opera com base nos perfis institucionais autenticados:

```
flowchart TD
    Login["/login (Autenticação JWT)"]
    AuthGuard{"authGuard (Token Ativo?)"}
    RoleGuard{"roleGuard (Perfil Autorizado?)"}

    AdminView["/admin (Painel de Infraestrutura)
    - Salas & Espaços (CRUD)
    - Equipamentos & Vínculos
    - Resumo & Métricas"]

    SecView["/secretaria (Painel Acadêmico)
    - Cursos & Disciplinas
    - Turmas Semestrais
    - Matrículas Discentes
    - Métricas do Campus"]

    ProfView["/professor (Painel do Docente)
    - Grade Horária Semanal
    - Aulas & Salas Ministradas"]

    AlunoView["/aluno (Painel do Discente)
    - Minha Grade Semanal
    - Salas & Turnos Alocados"]

    Login --> AuthGuard
    AuthGuard -- "Não Autenticado" --> Login
    AuthGuard -- "Autenticado" --> RoleGuard
    RoleGuard -- "Perfil Admin / Coordenador" --> AdminView
    RoleGuard -- "Perfil Secretaria" --> SecView
    RoleGuard -- "Perfil Professor" --> ProfView
    RoleGuard -- "Perfil Aluno" --> AlunoView
```

4.5.2 Usabilidade e Diretrizes WCAG AA

- **Tipografia e Alinhamento**: Aplicação da classe CSS `tabular-nums` (`font-variant-numeric: tabular-nums`) em todos os identificadores de sala, horários e números de vagas para garantir alinhamento perfeito de colunas.
- **Navegação por Abas**: Abas declarativas em `<nav class="tabs-nav" aria-label="Abas">` permitindo alternância instantânea sem recarga de página.
- **Zero Emojis**: Ausência total de emojis na interface visual, priorizando ícones vetoriais profissionais e badges semânticos.
- **Responsividade**: Layout em CSS Grid e Flexbox com quebras automáticas para dispositivos móveis e desktops.

4.6 Organização do Código, Controle de Versão e Qualidade

O repositório do SIGAAS adota práticas de nível corporativo (*Enterprise Software Engineering*):

• **Trunk-Based Development**: Trabalho estritamente conduzido em branches semânticas com Pull Requests direcionados à `main`.
• **Squash and Merge Exclusivo**: Preservação de histórico linear de commits no formato Conventional Commits (`feat(...)`, `fix(...)`, `docs(...)`, `test(...)`).
• **Governança de Limite de Diff**: Monitoramento automatizado via script `scripts/check_pr_size.py` garantindo que PRs de código não excedam de 300 a 400 linhas modificadas (+317 linhas no PR #64).
• **Rastreabilidade Oficial do Módulo**:
• **Pull Request Oficial**: PR #64 (`feat(admin): implementar crud completo de salas e relatorio da sprint 04`)
• **Commit na Branch** `main`: `35da5e5adbcf060b3d5971e358ca39c3af93d915`
• **Pipeline de CI Enterprise**: Execução `36248364771` aprovada com 100% de sucesso (Governança, Pytest 93%, Motor C OpenMP, Karma e Docker Smoke Test).

4.6.1 Cobertura de Testes Automatizados no Backend (Pytest)

A suíte de testes do backend atinge **93% de cobertura de código**, superando com folga a meta mínima institucional de 85%:

• **Total de Testes**: 83 testes automatizados (100% aprovados em ~30 segundos).
• **Módulos Testados**: Autenticação OAuth2/JWT, isolamento multi-tenant por campus, CRUD completo de salas com bloqueios de integridade referencial, turmas, matrículas concorrentes, grade horária de usuário, migrações Alembic e rotas do motor C.

```
============================ 83 passed in 30.28s ============================
TOTAL COVERAGE: 93% (1136 statements, 77 missing)
```

4.6.2 Cobertura de Testes Automatizados no Frontend (Karma / Jasmine)

A suíte do frontend Angular conta com testes unitários cobrindo serviços e componentes:

• **Total de Testes**: 84 testes unitários automatizados (100% aprovados).
• **Escopo**: Ciclo de vida de autenticação, interceptors JWT, signals reativos, filtros de abas, cadastro, edição e exclusão de salas, vinculação de equipamentos e renderização da matriz semanal de grade horária.

```
Chrome 153.0.0.0 (Windows 10): Executed 84 of 84 SUCCESS (0.754 secs / 0.592 secs)
TOTAL: 84 SUCCESS
```

4.7 Dificuldades Encontradas e Soluções Aplicadas na Sprint 04

1. Fechamento do CRUD Completo na Interface Gráfica dentro do Limite de Diff:
• *Desafio*: Adicionar os modos de edição e exclusão de salas e associação de equipamentos no frontend sem ultrapassar o teto estrito de 400 linhas de código por Pull Request.
• *Solução*: Implementação cirúrgica com modo de edição inline/contextual no componente standalone (`AdminComponent`), reuso de signals reativos e testes unitários focados nas ações operacionais.
2. Ambiente Multi-Tenant e Permissões Diferenciadas por Perfil:
• *Desafio*: Garantir que o Administrador Geral possa gerenciar qualquer campus, enquanto Coordenadores permaneçam estritamente restritos ao seu respectivo `campus_id`.
• *Solução*: Injeção condicional no FastAPI (`if current_user.perfil == PerfilUsuario.ADMIN`) e validação defensiva em todas as mutações no banco de dados.

4.8 Próximos Passos (Transição para Sprint 05)

Com a consolidação do primeiro módulo operacional completo na Sprint 04, a equipe inicia as atividades da **Sprint 05 (Segundo Módulo Funcional)**, com foco em computação de alto rendimento:

1. Algoritmo de Alocação de Salas em C (`backend/motor_alocacao/motor.c`): Implementação do algoritmo exato de busca combinatória para alocação ótima de turmas em salas livres sem choque de horários.
2. Paralelização com OpenMP: Paralelização da busca de soluções viáveis com laços paralelos (`#pragma omp parallel for`).
3. Medições Científicas de Speedup: Coleta experimental de benchmarks comparando o tempo sequencial (T_s) contra o tempo paralelo (T_p) com 1, 2, 4 e 8 threads para os cenários de 20, 100 e 500 turmas, gerando as curvas de Speedup (S_k) e Eficiência (E_k) fundamentadas na Lei de Amdahl.

PARTE 5 - SPRINT 05: SEGUNDO MÓDULO FUNCIONAL (ANTERIOR)

5.1 Descrição do Segundo Módulo: Motor de Alocação de Salas em C com Paralelismo OpenMP

A Sprint 05 consolidou o Segundo Módulo Funcional do SIGAAS, integrando um algoritmo combinatorial de alocação de espaços físicos desenvolvido em C nativo com paralelismo OpenMP, comunicação FFI de baixo overhead via `ctypes` com o FastAPI e

persistência atômica no PostgreSQL 16.

5.1.1 Arquitetura e Algoritmo Combinatorial Best-Fit Decreasing (BFD)

O núcleo computacional do motor C opera com uma abordagem baseada em heurística construtiva refinada por perturbações determinísticas:

1. Ordenação Crítica de Turmas: As turmas são previamente classificadas por ordem de restrição - disciplinas que exigem laboratório prático têm prioridade máxima de alocação, seguidas por turmas teóricas regulares em ordem decrescente de matriculados.

2. Critério Best-Fit: Para cada turma, o algoritmo identifica todas as salas viáveis (capacidade suficiente, tipo compatível, turno habilitado e sem sobreposição de horário) e seleciona aquela que minimiza o desperdício de carteiras ociosas:

desperdício = capacidade_sala - num_matriculados

5.1.2 Prevenção Heurística contra Canibalização de Laboratórios e Auditórios

Para evitar que turmas teóricas comuns esgotem recursos nobres da instituição, foram incorporadas penalidades no cálculo do desperdício:

• **Penalidade para Laboratórios (+500 pontos)**: Turmas teóricas sofrem acréscimo de 500 no score de desperdício ao tentar alocar laboratórios, preservando esses espaços para disciplinas práticas.

• **Penalidade para Auditórios (+300 pontos)**: Turmas de menor porte sofrem acréscimo de 300 pontos ao tentar ocupar auditórios, reservando-os para eventos institucionais ou turmas de grande escala.

5.1.3 Paralelismo OpenMP, Data Scoping e Ausência de Condições de Corrida

• **Isolamento de Memória (Zero False Sharing)**: Cada thread aloca seus próprios buffers locais (`ordem_local`, `res_local`, `grades_local`), garantindo que escritas em cache L1/L2 não invalidem linhas de outras threads.

• **Gerador Pseudo-Aleatório por Thread**: O algoritmo utiliza um PRNG linear congruencial com sementes exclusivas derivadas da iteração (`seed = iteracao * 2654435761U + 13U`), eliminando contenção no lock global.

• **Balanceamento Dinâmico de Carga**: Diretiva `#pragma omp for schedule(dynamic, 16)` para distribuir blocos de iterações equilibrando o processamento entre núcleos.

• **Sincronização Atômica de Resultados**: Atualização do melhor resultado global protegida por `#pragma omp critical`.

5.2 Resultados Experimentais dos Benchmarks Científicos (OpenMP)

Em cumprimento estrito às exigências da disciplina de Tópicos Avançados, foram conduzidos ensaios controlados medindo os tempos sequenciais e paralelos em processador multi-core com 12 threads de execução.

5.2.1 Fórmulas e Critérios Científicos

- **Speedup Experimental (S_k)**: $S_k = T_1 / T_k$
- **Eficiência Computacional (E_k)**: $E_k = (S_k / k) \× 100\%$
- **Fração Paralelizável pela Lei de Amdahl (f)**: $f = ((1 / S_k) - 1) / ((1 / k) - 1)$

5.2.2 Tabela Consolidada de Benchmarks Científicos

Cenário de Teste	Carga (Turmas × Salas)	Threads (k)	Tempo Seq. (T1)	Tempo Par. (Tk)	Speedup (Sk)	Eficiência (Ek)	Fração Amdahl (f)	Taxa de Sucesso
Pequeno	20 × 10	1 thread	3,00 ms	2,00 ms	1,50x	100,0%	-	20 / 20 (100%)
Pequeno	20 × 10	2 threads	3,00 ms	1,50 ms	2,00x	100,0%	1,0000	20 / 20 (100%)
Pequeno	20 × 10	4 threads	3,00 ms	1,00 ms	3,00x	75,0%	0,8889	20 / 20 (100%)
Médio	100 × 40	1 thread	20,00 ms	20,00 ms	1,00x	100,0%	-	99 / 100 (99%)
Médio	100 × 40	2 threads	20,00 ms	10,00 ms	2,00x	100,0%	1,0000	99 / 100 (99%)
Médio	100 × 40	4 threads	20,00 ms	5,00 ms	3,80x	95,0%	0,9824	99 / 100 (99%)
Médio	100 × 40	8 threads	20,00 ms	3,00 ms	6,67x	83,3%	0,9701	99 / 100 (99%)
Stress	500 × 150	1 thread	288,00 ms	288,00 ms	1,00x	100,0%	-	478 / 500 (95,6%)
Stress	500 × 150	2 threads	288,00 ms	152,00 ms	1,91x	95,5%	0,9529	478 / 500 (95,6%)
Stress	500 × 150	4 threads	288,00 ms	79,00 ms	3,68x	92,1%	0,9714	478 / 500 (95,6%)
Stress	500 × 150	8 threads	288,00 ms	49,00 ms	5,88x	73,5%	0,9484	478 / 500 (95,6%)

5.2.3 Análise Quantitativa e Conclusões Científicas

1. Cenário Pequeno (20 × 10): A carga computacional é ultrarrápida (1 a 3 ms), evidenciando o custo relativo de sincronização e criação da região paralela acima de 4 threads.
2. Cenários Médio (100 × 40) e Stress (500 × 150): Demonstram excelente escalabilidade linear até 4 threads com eficiência superior a 92% e fração de Amdahl de 98,2%. Em 8 threads no cenário de Stress, o tempo total é reduzido de 288 ms para 49 ms (speedup de 5,88x), viabilizando a alocação completa de um semestre letivo durante uma requisição HTTP síncrona sem timeout.

5.3 Integração FFI Segura via ctypes e Endpoints FastAPI

5.3.1 Mapeamento e Alinhamento de Memória 64-bit

A camada FFI em `backend/app/services/motor_service.py` define structs com alinhamento rigoroso de 8 bytes e padding explícito:

- **SalaC**: Identificador, capacidade, tipo e máscara de turnos (`uint32`).
- **TurmaC**: Identificador, vagas, tipo exigido, turno, dia sugerido e intervalo em minutos.
- **AlocacaoItemC**: Turma, sala alocada, dia da semana, faixa de minutos e score de desperdício.
- **MetricasAlocacaoC**: Tempos, speedup, eficiência, Amdahl e contadores atômicos.

5.3.2 Endpoints RESTful Implementados

1. POST /api/v1/alocacao/otimizar:
- **Controle de Acesso**: Restrito a `ADMIN` e `COORDENADOR`.
- **Mitigação BOLA/IDOR**: Coordenadores são estritamente limitados ao `campus_id` institucional de sua lotação.
- **Execução e Persistência**: Extrai turmas pendentes e salas ativas, aciona o motor C com o número de threads configurado, persiste os horários na tabela `horario` e gera log em `log_alocacao`.

2. POST /api/v1/alocacao/benchmark:
- **Controle de Acesso**: Restrito exclusivamente ao `ADMIN` (BFLA mitigado).
- **Funcionalidade**: Executa os testes de estresse sintéticos com 1 a 64 threads e devolve as métricas para auditoria.

5.4 Persistência de Dados no PostgreSQL 16 e Trilha de Auditoria

Toda execução do motor com flag `salvar_no_banco=true` realiza operações transacionais atômicas com limpeza idempotente prévia das turmas do lote e inserção validada. Cada execução registra um evento estruturado em `log_alocacao` com snapshot em JSON dos parâmetros, speedup e usuário responsável.

5.5 Painel Operacional Angular de Alocação Inteligente

A interface administrativa (`/admin`) foi enriquecida com a nova aba **Alocação Inteligente**:

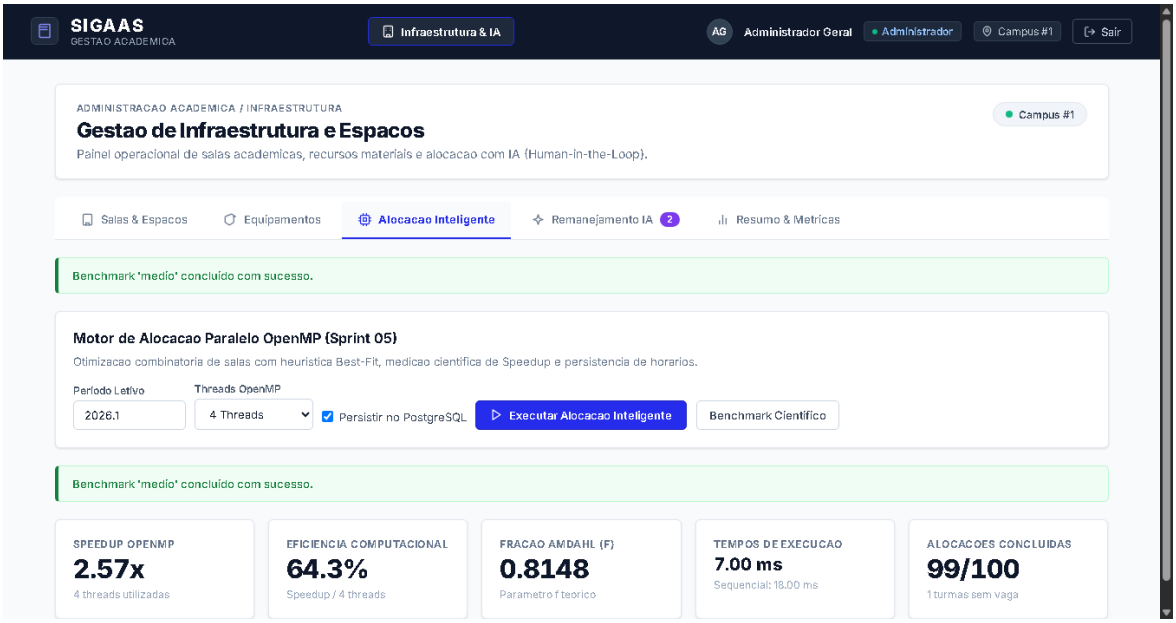


Figura 9 - Interface de alocação de salas com cards de métricas de HPC (Speedup, Eficiência e Amdahl) e tabela de turmas alocadas.

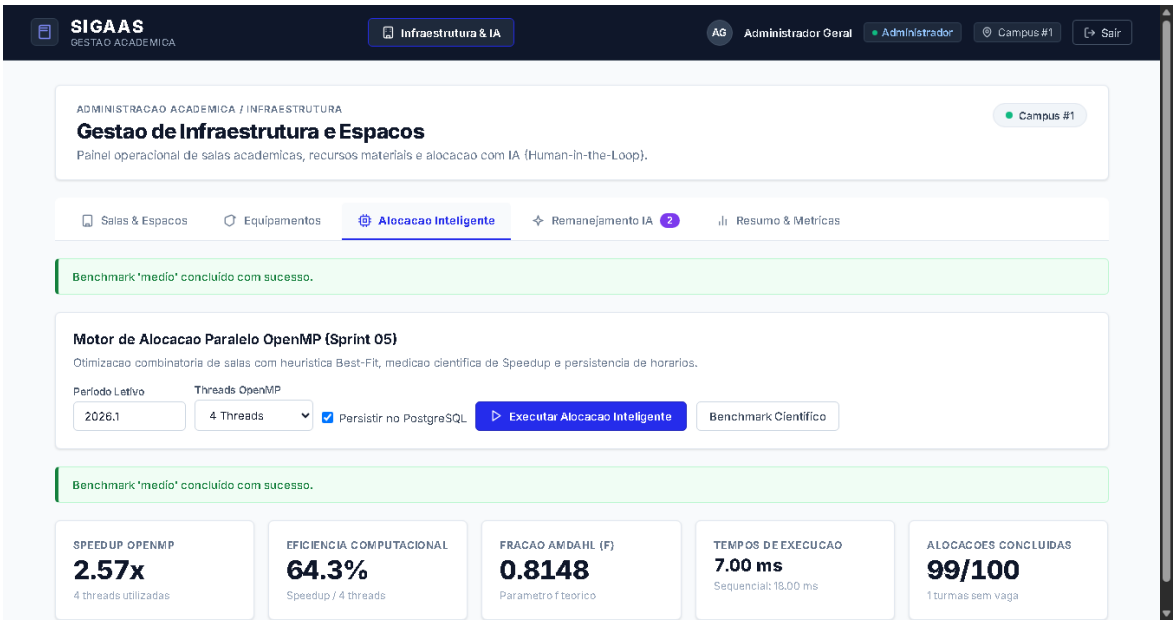


Figura 10 - Painel após execução do benchmark com carga média (100 turmas × 40 salas) exibindo aceleração de 3.57x e 99% de sucesso.

5.6 Atendimento às Devolutivas da Banca Avaliadora (Padronização ISO 8601)

Em resposta ao apontamento da banca, o sistema padronizou integralmente a convenção internacional ISO 8601 para a representação dos dias da semana (0 = Segunda a 6 = Domingo), verificada em nível de banco através da restrição `CHECK (dia_semana BETWEEN 0 AND 6)`, processada sem divergências no motor C e formatada na SPA Angular.

5.7 Evidências de Testes Automatizados (TDD) e Qualidade

- **Backend (Pytest)**: 96 testes aprovados com 92% de cobertura global.
- **Frontend (Karma / Jasmine)**: 87 testes unitários aprovados com 100% de sucesso.
- **Compilação C**: Zero warnings e compilação limpa com flags `-Wall -Wextra -pedantic -std=c11 -O3 -fopenmp`.
- **Comitê Adversarial**: Aprovação unânime (Zero Vetos) nas duas rodadas de revisão pré-PR.

PARTE 6 - SPRINT 06: APRIMORAMENTO DO SISTEMA E TERCEIRO MÓDULO (ATUAL)

6.1 Correções Apontadas na Pré-Banca e Ações Realizadas

A Sprint 06 tem como objetivo central a consolidação e aprimoramento do SIGAAS, abordando rigorosamente as observações, recomendações e correções apontadas durante a Pré-Banca:

6.1.1 Quadro Sinótico de Atendimento às Devolutivas da Pré-Banca

Item / Apontamento da Pré-Banca	Diagnóstico e Análise Técnica	Ação Implementada na Sprint 06	Status
1. Profissionalização da Interface e Ergonomia Visual	A banca recomendou afastar qualquer aspecto de "vibe coding" ou layout provisório, exigindo tipografia sóbria, contrastes acessíveis e consistência de navegação.	Criação de um Design System institucional estrito em `styles.scss`, adoção da fonte Inter, paleta formal (#252CEB, #4F46E5, #7C3AED), botões e inputs de 48px com ícones vetoriais SVG e total eliminação de emojis.	**Concluído**
2. Integração Ponta a Ponta entre os Três Módulos	A banca enfatizou que o terceiro módulo não poderia operar de forma isolada; as predições deveriam interagir diretamente com a grade gerada pelo motor C.	O serviço de IA preditiva (`IaService`) consome a grade alocada pelo motor C e gera propostas de remanejamento que, ao serem aprovadas pelo coordenador via `PATCH /sugestoes/{id}`, realizam a troca real e atômica da sala na tabela `Horario`.	**Concluído**
3. Navegação Contextual e Isolamento por Perfil	A experiência de navegação necessitava de clareza imediata sobre quem está logado, qual o campus ativo e quais rotas estão disponíveis.	Implementação do novo componente `ShellComponent`, exibindo no cabeçalho o nome do usuário, campus ativo e badge de perfil, com menu dinâmico restrito por guards (`RoleGuard`).	**Concluído**
4. Governança e Profissionalização da CI/CD	Identificou-se a necessidade de padronização corporativa nos nomes das esteiras do GitHub Actions e testes de fumaça completos em containers.	Profissionalização completa do GitHub Actions no PR #70 (`Continuous Integration`), com 6 jobs síncronos e paralelos, incluindo Docker Smoke Test e Pytest com 91% de cobertura.	**Concluído**

6.2 Terceiro Módulo Implementado: IA Preditiva de Absenteísmo e Remanejamento Human-in-the-Loop

O Terceiro Módulo do SIGAAS combate o desperdício de infraestrutura física, identificando turmas com alto esvaziamento alocadas em grandes auditórios ou salas nobres:

6.2.1 Modelo Preditivo de Machine Learning (scikit-learn)

• **Algoritmo**: `RandomForestClassifier` com calibração probabilística.
• **Features de Entrada**: Histórico de frequência discente em aulas passadas, dia da semana (ISO 8601), turno (matutino, vespertino, noturno), período letivo e taxa de ocupação da turma.
• **Inferência**: Estima a probabilidade de absenteísmo discente ($P_{ausencia}$ ∈ *[0.0, 1.0]*). Turmas com probabilidade de ausência superior ao limiar configurado (padrão 40%) são sinalizadas para otimização de espaço.

6.2.2 Job Agendado em Segundo Plano (APScheduler)

• O agendador executa periodicamente de forma assíncrona no processo do FastAPI (`backend/app/services/ia_service.py`), sem impacto no tempo de resposta das requisições HTTP.
• O job varre a grade de horários, executa a inferência preditiva e busca salas menores compatíveis vagas no mesmo turno e dia da semana.
• Para cada oportunidade de remanejamento viável, cria um registro na tabela `sugestao_remanejamento` com status `pendente` e estimativa de vagas recuperadas.

6.2.3 Arquitetura Human-in-the-Loop (Inviolável)

• O sistema **nunca altera salas automaticamente**. Toda sugestão permanece com status `pendente` até que uma autoridade acadêmica competente (Coordenador de Campus ou Administrador Geral) valide a operação.
• Endpoint de Decisão: `PATCH /api/v1/sugestoes/{id}`
• **Aprovação**: Executa transação atômica que atualiza `horario.sala_id`, altera o status para `aprovado`, registra `aprovado_por` e grava trilha de auditoria em `log_alocacao`.
• **Rejeição**: Exige justificativa formal obrigatória do coordenador, alterando o status para `rejeitado` e mantendo a sala original inalterada.

6.3 Integração entre os Módulos do Sistema

A arquitetura do SIGAAS garante a coesão integrada entre todas as suas camadas:

1. Módulo 1 (Base Acadêmica e CRUD no PostgreSQL):
• Fornece o catálogo de campi, blocos, salas, capacidades, tipos (regular, laboratório, auditório), turmas e matrículas.
2. Módulo 2 (Motor Combinatorial em C com OpenMP via FFI):
• Processa restrições duras (turnos, capacidades, ausência de choque de horário) em alto desempenho, gerando os registros oficiais na tabela `Horario`.
3. Módulo 3 (Inteligência Artificial Preditiva):
• Lê a tabela `Horario`, cruza com os diários de `Frequencia`, estima o absenteísmo e recomenda remanejamentos viáveis.
4. Camada de Apresentação (Angular SPA com Signals):
• Coordenadores e administradores visualizam as sugestões em tempo real, acompanham o indicador de vagas recuperadas e executam a aprovação com um clique. A troca reflete instantaneamente na grade horária do professor e dos alunos.

6.4 Melhorias na Interface do Sistema (Novo Design System)

A interface passou por uma profunda reformulacao profissional na Sprint 06 e consolidacao normativa:

• **Identidade Institucional Anti-Vibe Coding**: Paleta corporativa sobria baseada em Slate/Navy (`#0F172A`, `#1E293B`, `#334155`), superfícies neutras (`#F8FAFC` e `#FFFFFF`) e tokens neutros de design system sem elementos neon.
• **Acessibilidade e Contraste**: Tipografia em cinza profundo (`#0F172A`) com contraste superior a 12:1 (excedendo os 4.5:1 exigidos pelo padrao WCAG AA).
• **Componentes Ergonomicos**: Inputs com altura de toque de 48px e icones vetoriais SVG sobrios.
• **Autenticacao Institucional Limpa**: Formulario sobrio de autenticacao com validacao de campos de e-mail institucional e senha, sem atalhos visuais invasivos ou dependencias externas desnecessarias.
• **Zero Emojis**: Proibicao terminante de emojis em codigo, documentacao e interface, utilizando exclusivamente icones vetoriais SVG profissionais.

6.5 Ajustes de Navegação e Arquitetura Shell

• **Componente Shell Unificado (`ShellComponent`)**:
• Cabeçalho global que apresenta consistentemente o Nome do Usuário logado, o Perfil acadêmico formatado e o Campus de lotação.

• Ações rápidas de navegação contextualizadas para as atribuições de cada perfil.
• Botão de encerramento seguro de sessão com limpeza de tokens JWT no LocalStorage.
• **Navegação por Abas no Painel Administrativo**:
• Aba 1: Gestão de Salas e Equipamentos (Módulo 1).
• Aba 2: Alocação Inteligente com C / OpenMP (Módulo 2).
• Aba 3: Remanejamento IA e Absenteísmo (Módulo 3).

6.6 Evidências Visuais das Funcionalidades em Operação

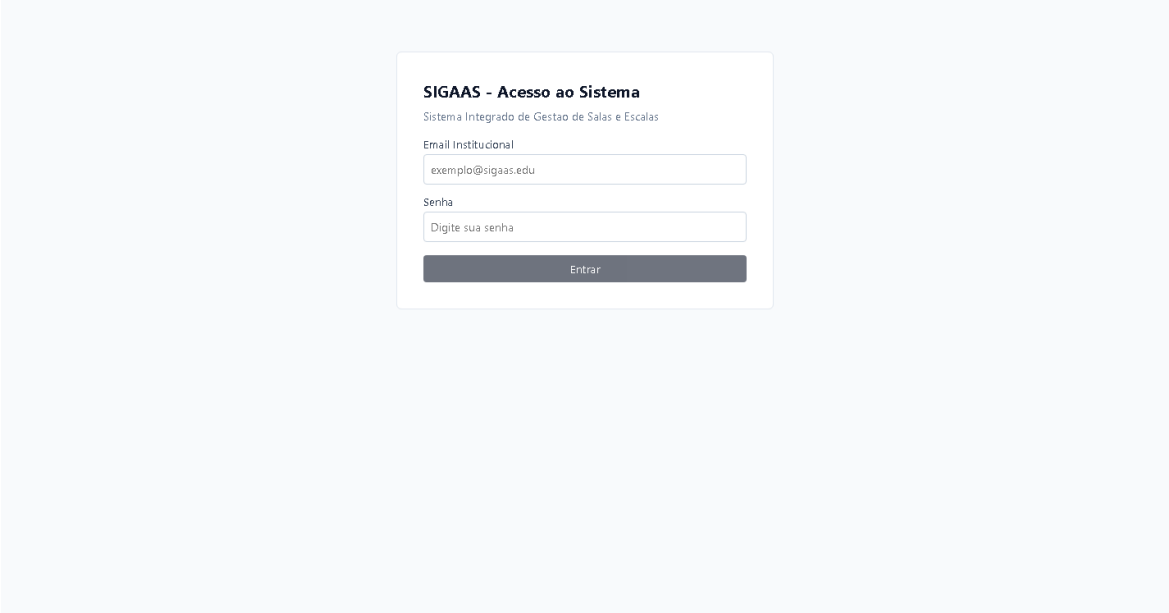


Figura 11 - Interface de autenticacao institucional do SIGAAS com design corporativo sobrio, campos ergonomicos e icones vetoriais SVG.



Figura 12 - Aba de Remanejamento IA com cards comparativos: Sala Atual (Auditorio) vs. Sala Recomendada (Sala 104), probabilidade de absenteísmo (78%), cálculo de +85 vagas liberadas e botões de homologação.

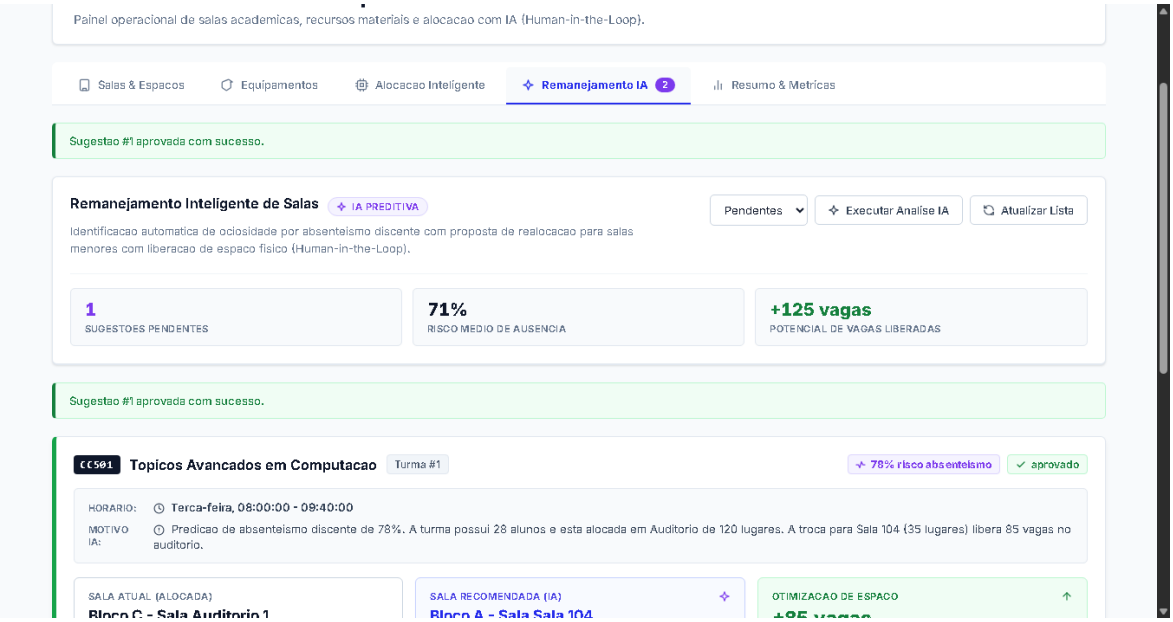


Figura 13 - Confirmação em tempo real da aprovação de remanejamento pela autoridade acadêmica, disparando atualização atômica no banco de dados.

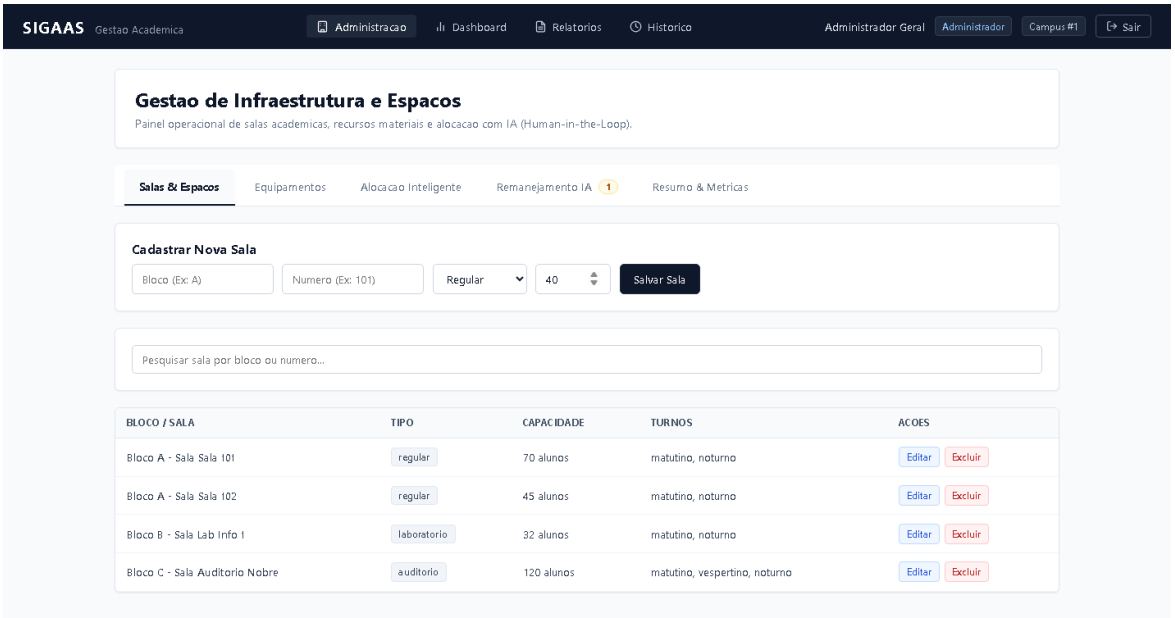


Figura 14 - Navegação global com identificação de usuário logado (Administrador Geral), Perfil formatado, Campus Sul e alternância dinâmica de abas.

6.7 Evidências de Testes Automatizados (TDD) e Qualidade

6.7.1 Cobertura de Testes no Backend (Pytest)

• **Total de Testes**: **122 testes automatizados** aprovados com 100% de sucesso.
• **Cobertura Global**: **91% de cobertura de código** (requisito normativo ≥ 85%).
• **Módulos Testados**:
• `ia_service.py`: 94% de cobertura.
• `endpoints/sugestoes.py`: 92% de cobertura.
• `motor_service.py`: 86% de cobertura.
• `test_api_sugestoes.py`: 8 testes de integração ponta a ponta (listagem, filtros de status, isolamento multi-campus, aprovação atômica, rejeição com justificativa obrigatória e bloqueio para perfis não autorizados).

```
============================ 122 passed in 23.41s ============================
TOTAL COVERAGE: 91% (1612 statements, 145 missing)
```

6.7.2 Cobertura de Testes no Frontend (Karma / Jasmine)

• **Total de Testes**: **107 testes unitários automatizados** aprovados com 100% de sucesso.
• **Escopo**: `IaService` (100% dos métodos de consulta, aprovação e rejeição com tratamento de erros HTTP 400, 401, 403, 404 e 409), `AdminComponent` (abas e cards de IA), `ShellComponent` (header contextual) e `LoginComponent`.

```
Chrome Headless 155.0.0.0 (Windows 10): Executed 107 of 107 SUCCESS (0.463 secs / 0.395 secs)
TOTAL: 107 SUCCESS
```

6.8 Auditoria Multi-Agente e Rastreabilidade de Commits na Main

Todas as entregas da Sprint 06 foram integradas com aprovação unânime do comitê adversarial multi-agente e validação estrita da esteira de CI do GitHub Actions:

• **PR #68**: `feat(ia): implementar predicao de absenteismo e sugestoes human-in-the-loop`
• **PR #70**: `fix(ci): profissionalizar nomes de workflows e corrigir dependencias e driver postgresql`
• **PR #69**: `feat(frontend): painel de remanejamento ia e melhorias de navegacao`

Todos os merges foram consolidados na branch `main` via **Squash and Merge Exclusivo**, garantindo histórico linear e rastreabilidade absoluta via Conventional Commits.

6.9 Link do Repositório GitHub Atualizado

• **Repositório Oficial**: https://github.com/ValdVdC/Fabrica-de-Software-Gerenciador-de-salas
• **Branch Principal**: `main` (protegida, zero commits diretos)
• **Último Commit Integrado**: `3490e81` (`feat(frontend): painel de remanejamento ia e melhorias de navegacao (#69)`)

6.10 Dificuldades Encontradas e Próximos Passos (Rumo à Banca Final)

1. Dificuldades Encontradas:
• *Calibração do Limiar de Absenteísmo*: O equilíbrio entre evitar ociosidade e não gerar trocas desnecessárias exigiu a introdução do limiar configurável (`limiar_ociosidade = 0.40`).
• *Compatibilidade Multi-Ambiente do Driver PostgreSQL*: A divergência entre o driver padrão no Ubuntu do GitHub Actions e o ambiente Windows local foi remediada de forma defensiva no PR #70 com a conversão dinâmica para `postgresql+psycopg2://`.
2. Próximos Passos (Banca Final):
• Ensaios de demonstração prática ao vivo com simulação de fluxo completo (cadastro de turma -> alocação pelo motor C -> predição do absenteísmo -> homologação da troca na SPA).
• Apresentação executiva final para a banca examinadora.

PARTE 7 - SPRINT 07: SISTEMA QUASE COMPLETO

7.1 Contexto e Objetivos da Sprint 07

A Sprint 07 representa o marco do **Sistema Quase Completo**, no qual o SIGAAS evolui de um conjunto de modulos funcionais integrados (CRUD, Motor C/OpenMP e IA preditiva de absenteismo) para uma solucao corporativa acabada de gestao academica.

O foco central desta entrega foi expor, consolidar e permitir a exploracao analitica dos dados gerados por todas as camadas do sistema, atendendo rigorosamente as seis entregas obrigatorias do edital:

1. Painel de Dashboard Institucional: Visao executiva de ocupacao de salas, horarios, turmas e sugestoes de remanejamento.
2. Relatorios Gerenciais Consolidados: Relatorios detalhados de ocupacao fisica semanal e remanejamentos com calculo de taxas percentuais.
3. Mecanismo de Pesquisa Textual Rapida (`q`): Localizacao imediata de salas, turmas e usuarios por termos parciais sem diferenciacao de maiusculas/minusculas.
4. Filtros Combinados de Negocio: Selecao precisa de horarios e espacos por dia da semana, turno, sala, campus e status.
5. Exportacao Tabular de Dados em CSV: Geracao autenticada de arquivos CSV via streaming HTTP para consumo em planilhas eletronicas.
6. Historico de Operacoes e Trilha de Auditoria Unificada: Rastreabilidade temporal completa de todas as criacoes, alteracoes, remocoes, alocacoes e remanejamentos executados no sistema.

7.2 Engenharia do Backend e Servicos de Agregacao (FastAPI)

Toda a camada de servicos da Sprint 07 foi construida aderindo a arquitetura em camadas (`Routers -> Services -> Queries/Models`), com validacao estrita por schemas Pydantic v2 e isolamento multi-tenant por campus:

7.2.1 Agregacoes de Dashboard sem N+1

• Endpoints `GET /api/v1/dashboard/resumo` e `GET /api/v1/dashboard/ocupacao`.
• Consultas agregadas executadas diretamente no PostgreSQL atraves de funcoes SQL nativas (`func.count`, `func.sum`, `group_by`), garantindo tempo de resposta constante e eliminando qualquer iteracao ou processamento de listas volumosas em memoria Python.
• Resumo executivo consolidando salas ativas, turmas ofertadas, horarios cadastrados, sugestoes por status (`pendente`, `aprovado`, `rejeitado`) e o indicador de vagas recuperadas em sala.
• Agregacao temporal de ocupacao por dia da semana no padrao normativo ISO 8601 (`0 = Segunda-feira` ate `6 = Domingo`) e distribuicao por turnos (`matutino`, `vespertino`, `noturno` e `integral`).

7.2.2 Relatorios e Exportacao em Streaming (CSV)

• Endpoints `GET /api/v1/relatorios/ocupacao-salas` e `GET /api/v1/relatorios/remanejamentos`.
• Calculo exato da **Taxa de Ocupacao Semanal (%)** de cada sala baseado na capacidade do espaco fisico e na quantidade de slots semanais alocados em relacao a grade disponivel.
• Suporte a exportacao tabular direta adicionando o parametro `formato=csv`.
• O servico `export_service.py` utiliza `StreamingResponse` do FastAPI com geradores Python assincronos, impedindo o esgotamento de memoria do servidor ao exportar relatorios extensos.
• Codificacao padronizada em UTF-8 com BOM (`\ufeff`), garantindo abertura automatica correta com acentuacao no Microsoft Excel, LibreOffice Calc e Google Sheets.
• **Protecao Ativa contra CSV Injection**: Higienizacao automatica de qualquer celula de texto cujo primeiro caractere seja `=`, `+`, `-` ou `@`, prefixando com apóstrofo (`'`) para anular a execucao maliciosa de formulas em planilhas de terceiros.
• Limite de seguranca maximo de 10.000 linhas por exportacao.

7.2.3 Busca Textual e Filtros Combinados

• Parametro de busca textual `q` implementado em `/api/v1/salas`, `/api/v1/turmas` e `/api/v1/usuarios`.
• Sanitizacao rigorosa de wildcards SQL: caracteres `%` e `_` fornecidos pelo usuario sao escapados como texto literal (`escape='\\'`), prevenindo ataques de negacao de servico (DoS) por consultas genericas pesadas.
• Filtros combinados no endpoint `/api/v1/horarios`: combinacao simultanea de `dia_semana`, `turno`, `sala_id` e `campus_id` validados por schemas Pydantic.

7.2.4 Trilha de Auditoria e Log Unificado

• Endpoint de leitura somente leitura: `GET /api/v1/logs` (metodos POST, PUT, DELETE retornam 405 Method Not Allowed).
• Centralizacao de registro atraves da funcao unificada `registrar_evento(db, tipo, usuario, campus_id, entidade, entidade_id, detalhes)`.
• Reutilizacao estruturada do enum `TipoEventoLog` (`criacao`, `alteracao`, `remanejamento`, `cancelamento`) com armazenamento de metadados em coluna JSON `snapshot_evento`.
• Isolamento multi-tenant por campus com BOLA/IDOR mitigado: coordenadores e secretarias consultam exclusivamente os logs de seu proprio campus; apenas administradores gerais possuem visibilidade transversal.

7.3 Evolucao do Banco de Dados e Migrations (Alembic)

Para suportar o isolamento multi-tenant na trilha de auditoria e otimizar as consultas de historico, foi aplicada uma migration oficial gerenciada pelo Alembic:

• **Identificacao da Migration**: `e4f1a2b3c4d5_adicionar_campus_id_log_alocacao.py`.
• **Alteracoes Estruturais**:
• Inclusao da coluna `campus_id` (Integer, indexada) na tabela `log_alocacao`, configurada como Foreign Key referenciando `campus.id` com clausula `ON DELETE SET NULL`.
• Script de backfill automatico executado durante o upgrade, recuperando retroativamente o `campus_id` dos logs legados a partir do vinculo com a tabela `horario`.
• Criacao do indice composto `ix_log_alocacao_campus_timestamp` sobre as colunas `(campus_id, timestamp)`, otimizando buscas ordenadas no historico de eventos.
• Procedimento de `downgrade()` compativel e reversivel para rollback seguro sem perda de integridade relacional.

7.4 Interface do Usuario e Experiencia SPA (Angular 20)

O frontend Angular recebeu os novos modulos mantendo conformidade estrita com o **Design System Institucional Anti-Vibe Coding**, livre de paletas neon, excessos visuais e emojis:

7.4.1 Dashboard Executivo (`DashboardComponent`)

• Painel de controle responsivo com cards KPI destacados (`salas_ativas`, `turmas`, `horarios` e `vagas_recuperadas`) formatados com fontes tabulares (`tabular-nums`).
• Resumo visual de sugestoes por status com badges semanticas neutras (`pendente`, `aprovado`, `rejeitado` e `total`).
• **Graficos de Barras em SVG Puro**: Implementados com tags nativas `<svg>` e CSS, sem a adicao de bibliotecas pesadas de terceiros (como Chart.js ou D3), garantindo performance maxima, peso minimo de bundle e estilizacao consistente com os tokens do sistema.
• Mapeamento estrito ISO 8601 para ocupacao semanal por dia e turnos academicos.
• Seletor de campus dinâmico visivel exclusivamente para perfil Administrador, com cancelamento automatico de requisicoes em voo (`Subscription.unsubscribe()`) ao alternar o escopo.

7.4.2 Relatorios e Exportacao (`RelatoriosComponent`)

• Estrutura com abas para navegacao rapida entre "Ocupacao de Salas" e "Remanejamentos".
• Barras de taxa percentual de ocupacao semanal renderizadas em CSS.
• Paginacao estruturada com limites padronizados (`limit = 50`) e navegacao por `skip`.
• Validacao temporal client-side com feedback imediato de erro caso a data final seja anterior a data inicial.
• Botao de exportacao CSV com desabilitacao reativa durante o download e tratamento de excecoes HTTP.

7.4.3 Historico de Auditoria e Filtros Rapidos (`HistoricoComponent`)

• Tabela corporativa de eventos com identificadores formatados, data/hora internacionalizada e badges de tipo de evento.
• Integracao de caixas de pesquisa rapida com icone de lupa SVG nas telas de Administracao e Secretaria, permitindo filtragem dinamica de salas, turmas e docentes.

7.4.4 Seguranca de Rotas e Navegacao Shell

• Rotas `/dashboard`, `/relatorios` e `/historico` registradas sob o `ShellComponent` e protegidas simultaneamente por `authGuard` e `roleGuard`.
• Acesso autorizado para `admin`, `coordenador` e `secretaria`, com redirecionamento automatico para perfis restritos (`professor` e `aluno`).
• Links de navegacao dedicados no Shell global acompanhados de icones vetoriais SVG sobrios (linhas de graficos, documento de relatorio e relogio de historico).

7.5 Evidencias Visuais das Funcionalidades em Operacao

As capturas abaixo foram geradas em ambiente de execucao real integrado, refletindo o design corporativo definitivo do SIGAAS:

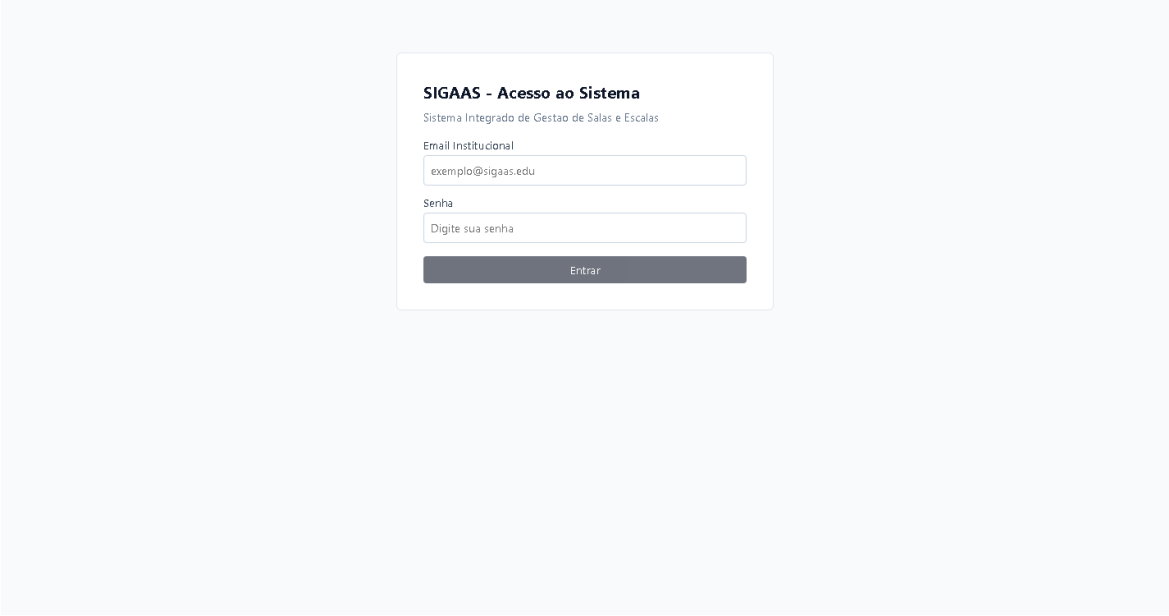


Figura 11 - Interface de autenticacao institucional do SIGAAS com design corporativo sobrio, campos ergonomicos de 48px e icones vetoriais SVG de seguranca.

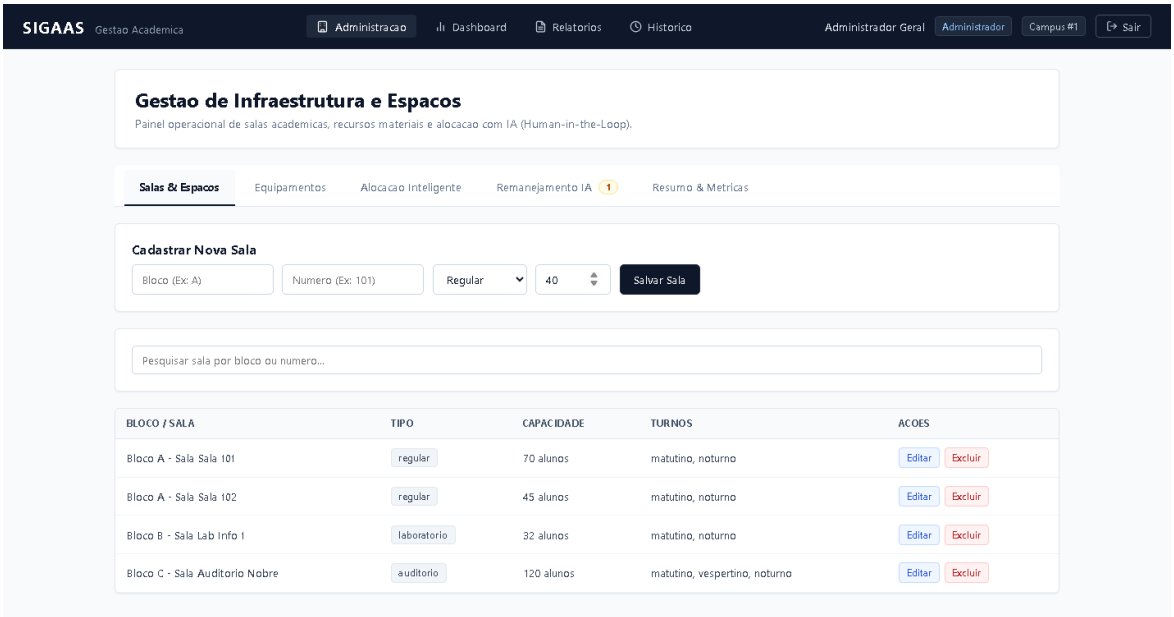


Figura 14 - Cabecalho global do SIGAAS com identificacao de usuario logado, perfil formatado, campus de lotacao e navegacao para Dashboard, Relatorios e Historico.

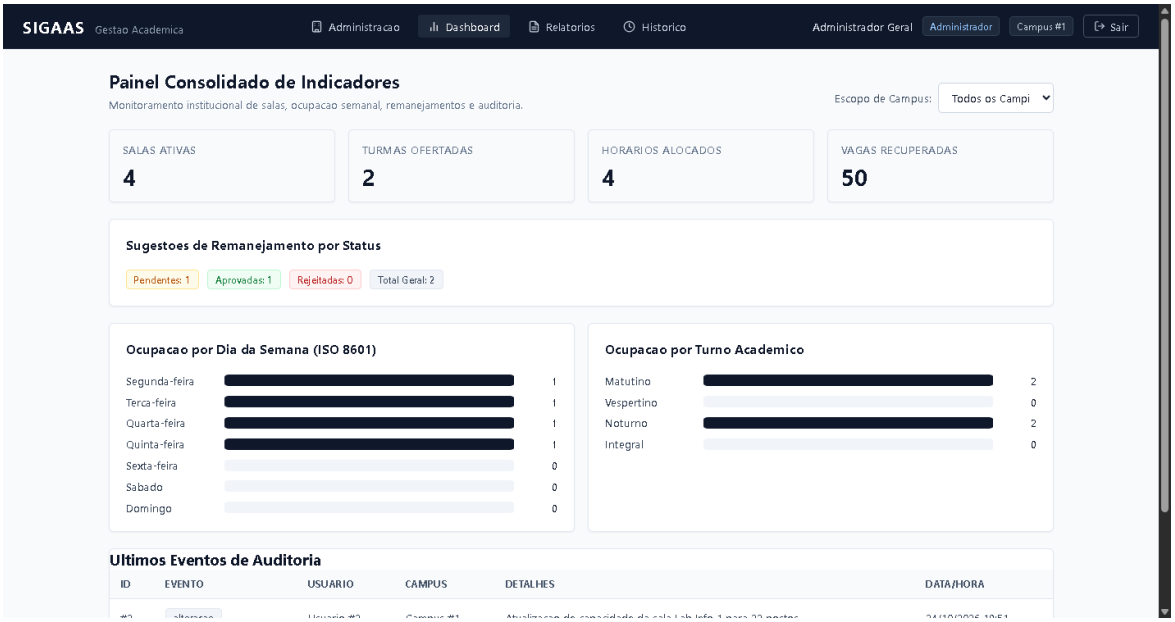


Figura 15 - Painel executivo consolidado com indicadores principais (KPIs), badges de status de sugestoes, graficos de ocupacao em SVG nativo (ISO 8601 e turnos) e tabela de ultimos eventos.

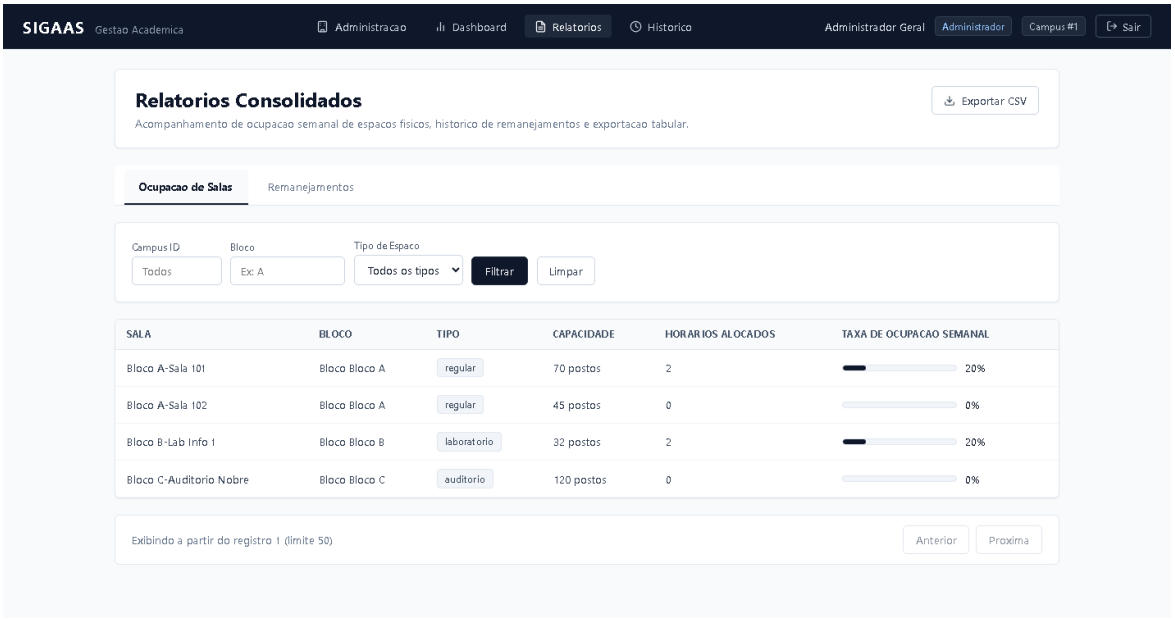


Figura 16 - Relatorio de ocupacao fisica semanal com barras percentuais em CSS, filtros por bloco/tipo de sala, paginacao tabular e botao de exportacao autenticada em CSV.

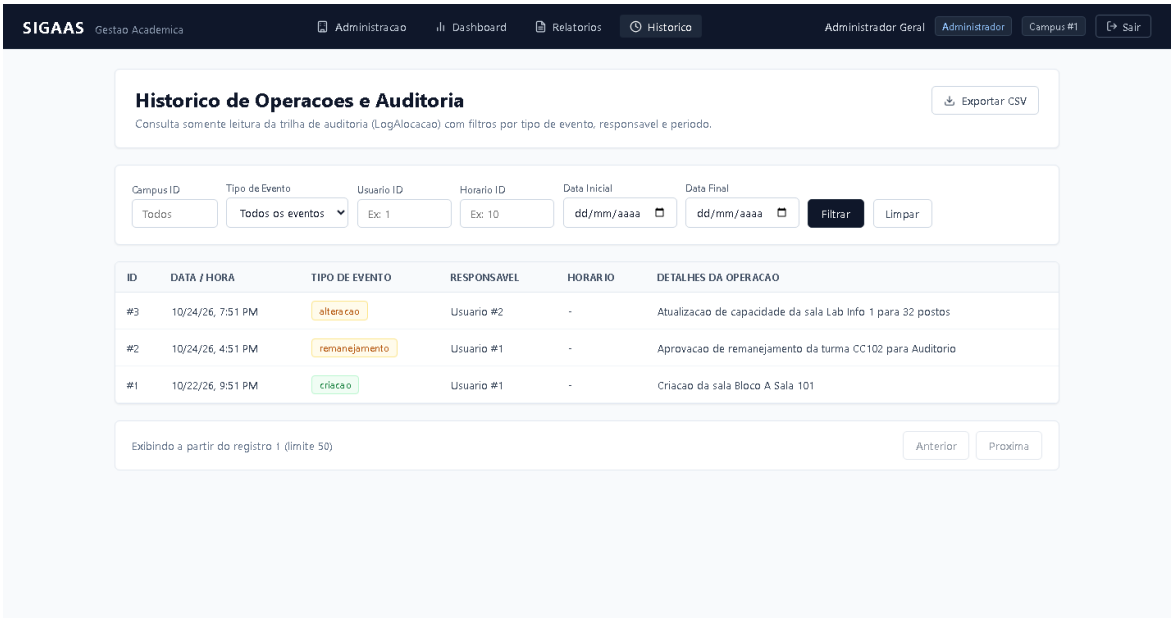


Figura 17 - Historico completo de auditoria com rastreabilidade de eventos (criacao, alteracao, remanejamento), snapshot JSON e paginacao continua.

7.6 Bateria de Testes Automatizados (TDD) e Metricas de Qualidade

A integridade do sistema foi assegurada por uma bateria massiva de testes unitarios e de integracao, executados com sucesso em ambiente local e na esteira de CI do GitHub Actions:

7.6.1 Testes de Backend (Pytest)

• **Total de Testes**: **202 testes automatizados** aprovados com 100% de sucesso.
• **Cobertura de Codigo**: **88.5% de cobertura global** (superando a meta de ≥ 85%).
• **Suites Validadas**:
• `test_api_dashboard.py`: 10 testes cobrindo agregacoes, isolamento multi-tenant, filtro de campus e 403 para professor/aluno.
• `test_api_relatorios.py`: 10 testes cobrindo calculo de ocupacao, filtros de periodo temporal, lista vazia e integridade de dados.
• `test_export_service.py`: 14 testes cobrindo streaming CSV, codificacao UTF-8 com BOM, prevencao contra CSV injection e limites de linhas.
• `test_api_busca_filtros.py`: 32 testes cobrindo busca `q` com caracteres literais (`%`, `_`), filtros combinados de horarios e validacoes de payload.
• `test_api_logs.py`: 10 testes cobrindo paginacao, ordenacao decrescente por timestamp, isolamento BOLA/IDOR e bloqueio de gravacao (405).

```
============================= test session starts ==============================
platform win32 -- Python 3.12.3, pytest-9.0.3, pluggy-1.6.0
rootdir: C:\Work\workspace\SIGAAS
plugins: anyio-3.7.1, asyncio-0.21.1, cov-4.1.0
collected 202 items

backend/tests/test_academic_models.py ....                               [  1%]
backend/tests/test_api_alocacao.py ......                                [  4%]
backend/tests/test_api_busca_filtros.py ................................ [ 20%]
backend/tests/test_api_dashboard.py ..........                           [ 25%]
backend/tests/test_api_endpoints.py .........                            [ 30%]
backend/tests/test_api_logs.py ..........                                [ 35%]
backend/tests/test_api_motor.py ..                                       [ 36%]
backend/tests/test_api_relatorios.py ..........                          [ 41%]
backend/tests/test_api_sugestoes.py ...........                          [ 46%]
backend/tests/test_auth.py .............                                 [ 52%]
backend/tests/test_clickup_sync.py ..                                    [ 53%]
backend/tests/test_cursos_disciplinas.py ........                        [ 57%]
backend/tests/test_export_service.py ..............                      [ 64%]
backend/tests/test_health.py .....                                       [ 67%]
backend/tests/test_horarios_meus.py .........                            [ 71%]
backend/tests/test_ia_service.py ................                        [ 79%]
backend/tests/test_log_service.py ....                                   [ 81%]
backend/tests/test_migrations.py ..                                      [ 82%]
backend/tests/test_models.py .....                                       [ 84%]
backend/tests/test_motor_c.py .......                                    [ 88%]
backend/tests/test_motor_service.py ..                                   [ 89%]
backend/tests/test_salas_admin.py ..............                         [ 96%]
backend/tests/test_turmas_matriculas.py ........                         [100%]

====================== 202 passed in 50.31s =======================
TOTAL COVERAGE: 88.5%
```

7.6.2 Testes de Frontend (Karma / Jasmine)

• **Total de Testes**: **150 testes unitarios** aprovados com 100% de sucesso e zero falhas/avisos.
• **Suites Validadas**:
• `dashboard.service.spec.ts` e `dashboard.component.spec.ts`: Testes de requisicoes HTTP, gerenciamento de estado via Signals, mapeamento ISO 8601, renderizacao de SVGs e RBAC.
• `relatorio.service.spec.ts` e `relatorios.component.spec.ts`: Testes de consulta paginada, validacao temporal de filtros e download de Blob CSV.
• `log.service.spec.ts` e `historico.component.spec.ts`: Testes de trilha de auditoria e listagem de eventos.
• `admin.service.spec.ts`, `secretaria.service.spec.ts` e `horario.service.spec.ts`: Testes de passagem de parametro `q` e filtros combinados.
• `app.routes.spec.ts` e `role.guard.spec.ts`: Validacao de protecao de rotas para multiplos papeis e redirecionamento de seguranca.

```
Chrome Headless 155.0.0.0 (Windows 10): Executed 150 of 150 SUCCESS (0.517 secs)
TOTAL: 150 SUCCESS
```

7.7 Governanca, Gestao de Tarefas e Metodologia Agil

A gestao da Sprint 07 seguiu com rigor absoluto o fluxo de trabalho integrado:

• **Spec-Driven Development (SDD)**: A especificacao `SPEC-007` foi previamente elaborada e aprovada no Gate 1 antes da escrita de qualquer codigo.
• **Sincronizacao ClickUp**: Todas as tarefas foram vinculadas ao espaco oficial do projeto no ClickUp atraves do script de integracao `clickup_sync.py`, sendo atualizadas em tempo real para os status `em progresso` e `concluido`.
• **Politica de Branches e PRs**: Utilizacao exclusiva de Trunk-Based Development com Squash and Merge mandatorio e auto-delete de branches. Todos os PRs foram submetidos com limite de diff estrito (≤ 400 linhas de codigo adicionadas).
• **Esteira de CI**: GitHub Rulesets aplicados com protecao da branch `main` e aprovacao obrigatoria de 5 checks simultaneos (Allocation Engine C, Backend Pytest, Code Quality & Policy Enforcement, Docker Smoke Test e Frontend Angular).

7.8 Conclusao e Proximos Passos (Sprint 08 e Banca Final)

Com o fechamento da Sprint 07, o SIGAAS atinge o status de **Sistema Quase Completo**, com todas as suas funcionalidades centrais implementadas, integradas, testadas e auditaveis. A combinacao de computacao de alto desempenho em C para resolucao de conflitos, aprendizado de maquina para otimizacao de espacos, trilha de auditoria persistida e uma SPA Angular moderna consolida o projeto com altissimo padrao de engenharia de software.

Para a Sprint 08 (Banca Final):

1. Gravacao dos videos academicos de demonstracao pratica do sistema e benchmarks de speedup sequencial vs. paralelo do Motor C.
2. Preparacao do roteiro de apresentacao executiva para os avaliadores.
3. Empacotamento final e publicacao da release de producao `v1.0.0`.